astronomy-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `astronomy_ai999x.about()` for the same answer.

Every capability astronomy-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import astronomy_ai999x

result = astronomy_ai999x.solve('distance_unit_conversion', d='3.08568e+16 m')
result.show()
# d_m = 3.08568e+16 m, d_km = 3.08568e+13, d_au = 206265, d_ly = 3.26157, d_pc = 1
```

By topic

topic	capabilities
Compact objects / black holes	Schwarzschild radius
Cosmology / angular scale	Angular size of an object at a redshift
Cosmology / distances	Cosmological distances at a redshift
Cosmology / Friedmann equation	Hubble parameter and critical density
Cosmology / redshift	Radial velocity from a Doppler redshift, Redshift and radial velocity from a spectral line
Cosmology / time	Lookback time to a redshift, Age of the universe at a redshift, Redshift at a given lookback time
Cosmology / volumes	Comoving volume out to a redshift
Distances and magnitudes / absolute magnitude	Absolute magnitude
Distances and magnitudes / distance modulus	Distance from the distance modulus
Distances and magnitudes / distance units	Astronomical distance units
Distances and magnitudes / magnitude scale	Flux ratio from a magnitude difference, Magnitude difference from a flux ratio, Combined magnitude of two stars
Distances and magnitudes / parallax	Distance from parallax
Distances and magnitudes / standard candles	Distance from luminosity and flux (inverse-square law)
Exoplanets / radial velocity	Radial-velocity semi-amplitude of a star
Exoplanets / transits	Transit depth, Transit duration

topic	capabilities
Gravitational lensing / point lens	Einstein radius of a point-mass lens
Observing / angular size	Angular size, Linear size from angular size
Observing / atmosphere	Airmass and atmospheric extinction
Orbits / binary stars	Masses of a visual binary star
Orbits / elliptical orbits	Speeds at periapsis and apoapsis
Orbits / Kepler's equation	Position on an elliptical orbit at a given time (Kepler's equation), Time of flight between two points of an elliptical orbit
Orbits / orbit determination	Orbital elements from position and velocity
Orbits / orbital manoeuvres	Hohmann transfer between circular orbits, Departure burn from a circular orbit, Delta-v of a combined plane change
Planets / atmospheres	Scale height of an isothermal atmosphere
Planets / equilibrium temperature	Equilibrium temperature of a planet
Plasma physics / frequencies	Plasma frequency, Upper hybrid frequency, Lower hybrid frequency
Plasma physics / lengths	Debye length, Inertial length (skin depth)
Plasma physics / particle motion	Gyrofrequency (cyclotron frequency), Gyroradius (Larmor radius)
Plasma physics / plasma parameter	Number of particles in a Debye sphere
Plasma physics / pressures	Plasma beta, Magnetic pressure
Plasma physics / velocities	Thermal speed of plasma particles
Plasma physics / waves	Alfven speed
Positional astronomy / coordinates	Altitude and azimuth from hour angle and declination, Angular separation of two sky positions, Position angle between two sky positions
Positional astronomy / meridian transit	Altitude of a star at culmination
Positional astronomy / rising and setting	Rising and setting of a star
Stars / bolometric magnitude	Absolute bolometric magnitude from luminosity, Luminosity from absolute bolometric magnitude, Apparent bolometric magnitude and flux
Stars / effective temperature	Effective temperature of a star
Stars / luminosity	Luminosity of a star (Stefan-Boltzmann)
Stars / radius	Radius of a star from luminosity and temperature
Stars / star formation	Free-fall time of a uniform cloud
Stars / stellar atmospheres	Ionization fraction of hydrogen (Saha equation)
Telescopes / imaging	Pixel scale and field of view of a telescope camera
Telescopes / light gathering	Light-gathering power of two telescopes
Telescopes / magnification	Telescope magnification, exit pupil and true field
Telescopes / resolution	Diffraction limit of a telescope (Rayleigh criterion)
Time / calendars	Julian date of a calendar date, Calendar date from a Julian date
Time / sidereal time	Greenwich and local mean sidereal time

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
distance_unit_conversion	Astronomical distance units	d	d_m, d_km, d_au, d_ly, d_pc
parallax_distance	Distance from parallax	p	d, d_pc, d_ly, p_arcsec
distance_from_distance_modulus	Distance from the distance modulus	m, M, A	d, d_pc, d_ly, mu
absolute_magnitude	Absolute magnitude	m, d, A	M, mu, d_pc
flux_ratio_from_magnitudes	Flux ratio from a magnitude difference	m1, m2	ratio, dm
magnitude_difference_from_flux_ratio	Magnitude difference from a flux ratio	ratio	dm
combined_magnitude	Combined magnitude of two stars	m1, m2	m, brightening
bolometric_magnitude_from_luminosity	Absolute bolometric magnitude from luminosity	L	M_bol, L_Lsun
luminosity_from_bolometric_magnitude	Luminosity from absolute bolometric magnitude	M_bol	L, L_Lsun
apparent_bolometric_magnitude	Apparent bolometric magnitude and flux	L, d	m_bol, F, M_bol
angular_size	Angular size	D, d	theta, theta_arcsec, theta_deg
linear_size_from_angle	Linear size from angular size	theta, d	D, D_km, D_au
stellar_luminosity	Luminosity of a star (Stefan-Boltzmann)	R, T	L, L_Lsun
stellar_radius	Radius of a star from luminosity and temperature	L, T	R, R_Rsun
effective_temperature	Effective temperature of a star	L, R	T
planet_equilibrium_temperature	Equilibrium temperature of a planet	T_star, R_star, a, A	T_eq, S
saha_hydrogen_ionization	Ionization fraction of hydrogen (Saha equation)	T, n	x, neutral, n_e
schwarzschild_radius	Schwarzschild radius	M	rs, rs_km, M_Msun
atmospheric_scale_height	Scale height of an isothermal atmosphere	T, mu, g	H, H_km
free_fall_time	Free-fall time of a uniform cloud	rho	t_ff, t_ff_yr
binary_star_masses	Masses of a visual binary star	P, a1, a2	M_total, M_total_Msun, M1_Msun, M2_Msun, a_au
hohmann_transfer	Hohmann transfer between circular orbits	r1, r2, M	dv1, dv2, dv, t_transfer, t_days, a_transfer
apsis_speeds	Speeds at periapsis and apoapsis	rp, ra, M	vp, va, a, e, P
escape_delta_v	Departure burn from a circular orbit	M, r, v_inf	dv, v_circ, v_depart
plane_change_delta_v	Delta-v of a combined plane change	v1, v2, di	dv

id	operation	inputs	outputs
orbit_from_state	Orbital elements from position and velocity	M, r, v, gamma	a, e, rp, ra, P, energy
kepler_position_at_time	Position on an elliptical orbit at a given time (Kepler's equation)	M, a, e, t	mean_anomaly, E, nu, r, v, P
time_of_flight	Time of flight between two points of an elliptical orbit	M, a, e, nu1, nu2	dt, dt_days, P
radial_velocity_semi_amplitude	Radial-velocity semi-amplitude of a star	M_star, m_p, P, e, i	K, a, a_au
transit_depth	Transit depth	Rp, Rs, b	depth, depth_ppm, k, grazing
transit_duration	Transit duration	P, a, Rs, Rp, i	T14, T14_h, T23, b
cosmological_distances	Cosmological distances at a redshift	z, H0, Om	D_C, D_C_Mpc, D_L_Mpc, D_A_Mpc, mu, D_L
angular_size_at_redshift	Angular size of an object at a redshift	z, H0, Om, D	theta_arcsec, scale_kpc_per_arcsec, D_A_Mpc
lookback_time	Lookback time to a redshift	z, H0, Om	t_L, t_L_Gyr
age_at_redshift	Age of the universe at a redshift	z, H0, Om	age, age_Gyr, age_now_Gyr
redshift_at_lookback_time	Redshift at a given lookback time	t, H0, Om	z, age_now_Gyr
comoving_volume	Comoving volume out to a redshift	z, H0, Om	V, V_Gpc3
recession_velocity_from_redshift	Radial velocity from a Doppler redshift	z	v, beta, v_cz
redshift_from_wavelengths	Redshift and radial velocity from a spectral line	lam_obs, lam_rest	z, v, beta, d_lambda
rayleigh_resolution	Diffraction limit of a telescope (Rayleigh criterion)	lam, D	theta, theta_arcsec
telescope_magnification	Telescope magnification, exit pupil and true field	f_obj, f_eye, D, afov	M, exit_pupil, exit_pupil_mm, tfov_deg
light_gathering_power	Light-gathering power of two telescopes	D1, D2	area_ratio, mag_gain
camera_pixel_scale	Pixel scale and field of view of a telescope camera	f, p, N	pixel_scale_arcsec, fov_deg, fov_arcmin
airmass_extinction	Airmass and atmospheric extinction	zen, k, m	X, alt_deg, extinction, m0
altaz_from_hadec	Altitude and azimuth from hour angle and declination	H, dec, lat	alt_deg, az_deg
culmination_altitude	Altitude of a star at culmination	dec, lat	alt_upper_deg, alt_lower_deg, circumpolar, never_rises
rise_set_hour_angle	Rising and setting of a star	dec, lat	H_set_h, time_up_sidereal_h, az_rise_deg, az_set_deg
angular_separation	Angular separation of two sky positions	ra1, dec1, ra2, dec2	sep_deg, sep_arcsec

id	operation	inputs	outputs
position_angle	Position angle between two sky positions	ra1, dec1, ra2, dec2	pa_deg
julian_date	Julian date of a calendar date	year, month, day, hour, minute, second	JD, MJD
calendar_date_from_julian_date	Calendar date from a Julian date	jd	year, month, day, hours
sidereal_time	Greenwich and local mean sidereal time	jd, lon	gmst_h, lmst_h
plasma_frequency	Plasma frequency	n, particle	omega_p, f_p
debye_length	Debye length	T, n	lambda_D, lambda_D_nm
debye_number	Number of particles in a Debye sphere	T, n	N_D, lambda_D
alfven_speed	Alfven speed	B, n	v_A, v_A_km_s
gyrofrequency	Gyrofrequency (cyclotron frequency)	B, particle	omega_c, f_c, T_c
gyroradius	Gyroradius (Larmor radius)	B, v_perp, particle	r_L, r_L_km, r_L_nm
thermal_speed	Thermal speed of plasma particles	T, convention, particle	v_th
plasma_beta	Plasma beta	T, n, B	beta, p_thermal, p_magnetic
magnetic_pressure	Magnetic pressure	B	p_B, energy_density, p_B_nPa
upper_hybrid_frequency	Upper hybrid frequency	B, n	omega_uh, f_uh
lower_hybrid_frequency	Lower hybrid frequency	B, n	omega_lh, f_lh
inertial_length	Inertial length (skin depth)	n, particle	d, d_km
einstein_radius	Einstein radius of a point-mass lens	M, D_l, D_s	theta_E, theta_E_arcsec, R_E, R_E_au
distance_from_flux	Distance from luminosity and flux (inverse-square law)	L, F	d, d_pc, d_ly
critical_density	Hubble parameter and critical density	H0, Om, z	H_z, rho_c, n_H

Astronomical distance units — `distance_unit_conversion`

A distance expressed in metres, kilometres, astronomical units, light-years and parsecs.

$$1\,\mathrm{pc} = \frac{648000}{\pi}\,\mathrm{au}, \quad 1\,\mathrm{ly} = c \times 365.25\,\mathrm{d}$$

input	meaning	condition
`d`	distance (m, km, au, ly, pc, kpc, Mpc ...)	

output	meaning
`d_m`	distance in metres
`d_km`	distance in kilometres
`d_au`	distance in astronomical units

output	meaning
`d_ly`	distance in light-years
`d_pc`	distance in parsecs

- **Method:** 1 $au = 149597870700$ m; 1 $pc = au 648000/pi$; 1 $ly = c 365.25$ days
- **Assumes:** $d > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('distance_unit_conversion', d=3.085677581491367e+16)
# -> d_m = 3.08568e+16 m, d_km = 3.08568e+13, d_au = 206265, d_ly = 3.26156, d_pc = 1   (SOLVED)
```

```
astronomy_ai999x.solve('distance_unit_conversion', d=149597870700.0)
# -> d_m = 1.49598e+11 m, d_km = 1.49598e+08, d_au = 1, d_ly = 1.58125e-05, d_pc = 4.84814e-06
(SOLVED)
```

```
astronomy_ai999x.solve('distance_unit_conversion', d=9460730472580800.0)
# -> d_m = 9.46073e+15 m, d_km = 9.46073e+12, d_au = 63241.1, d_ly = 1, d_pc = 0.306601   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Distance conversion: d = 1 pc	SOLVED	d_m = 3.08568e+16 m, d_km = 3.08568e+13, d_au = 206265, d_ly = 3.26156, d_pc = 1
Distance conversion: d = 1 ly	SOLVED	d_m = 9.46073e+15 m, d_km = 9.46073e+12, d_au = 63241.1, d_ly = 1, d_pc = 0.306601
Distance conversion: d = -3 ly	INVALID_INPUT	Astronomical distance units: the distance must be a positive number.
Distance conversion: d = 5	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: astronomy_ai999x.templates(); eve

Distance from parallax — `parallax_distance`

The distance of a star from its annual trigonometric parallax: d [pc] = $1/p$ [arcsec].

$$d\,[\mathrm{pc}] = \frac{1}{p\,[\mathrm{arcsec}]}$$

input	meaning	condition
`p`	annual parallax (arcsec, mas, rad, deg)	

output	meaning
`d`	distance
`d_pc`	distance in parsecs
`d_ly`	distance in light-years
`p_arcsec`	parallax in arcseconds

- **Method:** $d = 1/p$ with p in arcseconds gives parsecs
- **Assumes:** $p > 0$ (small-angle definition of the parsec)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('parallax_distance', p=4.84813681109536e-06)
# -> d = 3.08568e+16 m, d_pc = 1, d_ly = 3.26156, p_arcsec = 1   (SOLVED)
```

```
astronomy_ai999x.solve('parallax_distance', p=4.84813681109536e-07)
# -> d = 3.08568e+17 m, d_pc = 10, d_ly = 32.6156, p_arcsec = 0.1   (SOLVED)
```

```
astronomy_ai999x.solve('parallax_distance', p=3.7233690709212365e-06)
# -> d = 4.01781e+16 m, d_pc = 1.30208, d_ly = 4.24683, p_arcsec = 0.768   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Parallax distance: p = 0.1 arcsec	SOLVED	d = 3.08568e+17 m, d_pc = 10, d_ly = 32.6156, p_arcsec = 0.1
Parallax distance: p = 2 mas	SOLVED	d = 1.54284e+19 m, d_pc = 500, d_ly = 1630.78, p_arcsec = 0.002
Parallax distance: p = 0 arcsec	INVALID_INPUT	Distance from parallax: the parallax must be a positive number.
Parallax distance:	NEEDS_INPUT	parallax_distance needs p (annual parallax (arcsec, mas, rad, deg)). Example: solve('parallax_distance', p='4.84814e-06

Distance from the distance modulus — `distance_from_distance_modulus`

The distance of a star from its apparent and absolute magnitudes (and an optional extinction A): $m - M - A = 5$ log10($d/10$ pc).

$$d = 10^{(m-M-A)/5+1}\ \mathrm{pc}$$

input	meaning	condition
`m`	apparent magnitude	
`M`	absolute magnitude	
`A`	interstellar extinction in magnitudes (default 0)	

output	meaning
`d`	distance
`d_pc`	distance in parsecs
`d_ly`	distance in light-years
`mu`	distance modulus m - M - A

- **Method:** $d = 10$^((m - M - A)/5 + 1) pc
- **Assumes:** $A \geq 0$; $-10 < m - M - A < 60$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('distance_from_distance_modulus', m=10.0, M=0.0)
# -> d = 3.08568e+19 m, d_pc = 1000, d_ly = 3261.56, mu = 10   (SOLVED)
```

```
astronomy_ai999x.solve('distance_from_distance_modulus', m=15.0, M=5.0, A=0.0)
# -> d = 3.08568e+19 m, d_pc = 1000, d_ly = 3261.56, mu = 10   (SOLVED)
```

```
astronomy_ai999x.solve('distance_from_distance_modulus', m=7.0, M=2.0, A=1.0)
# -> d = 1.94693e+18 m, d_pc = 63.0957, d_ly = 205.791, mu = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Distance from distance modulus: m = 10, M = 0	SOLVED	d = 3.08568e+19 m, d_pc = 1000, d_ly = 3261.56, mu = 10
Distance from distance modulus: m = 7, M = 2, A = 1	SOLVED	d = 1.94693e+18 m, d_pc = 63.0957, d_ly = 205.791, mu = 4
Distance from distance modulus: m = 7, M = 2, A = -1	INVALID_INPUT	Distance from the distance modulus: the extinction A must not be negative.
Distance from distance modulus: m = 10	NEEDS_INPUT	distance_from_distance_modulus needs M (absolute magnitude). Example: solve('distance_from_distance_modulus', m=10, M=0)

Absolute magnitude — `absolute_magnitude`

The absolute magnitude of a star from its apparent magnitude and distance (and an optional extinction A).

$$M = m - 5\log_{10}\frac{d}{10\,\mathrm{pc}} - A$$

input	meaning	condition
`m`	apparent magnitude	
`d`	distance (pc, ly, m ...)	
`A`	interstellar extinction in magnitudes (default 0)	

output	meaning
`M`	absolute magnitude
`mu`	distance modulus 5 log10($d/10$ pc)
`d_pc`	distance in parsecs

- **Method:** $M = m - 5$ log10($d/10$ pc) - A
- **Assumes:** $d > 0$; $A \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('absolute_magnitude', m=12.0, d=3.085677581491367e+19)
# -> M = 2, mu = 10, d_pc = 1000   (SOLVED)
```

```
astronomy_ai999x.solve('absolute_magnitude', m=7.0, d=3.0856775814913674e+18)
# -> M = 2, mu = 5, d_pc = 100   (SOLVED)
```

```
astronomy_ai999x.solve('absolute_magnitude', m=8.0, d=3.0856775814913674e+18, A=1.0)
# -> M = 2, mu = 5, d_pc = 100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Absolute magnitude: m = 8, d = 100 pc, A = 1	SOLVED	M = 2, mu = 5, d_pc = 100
Absolute magnitude: m = 6.5, d = 10 pc	SOLVED	M = 6.5, mu = 0, d_pc = 10
Absolute magnitude: m = 6.5, d = 0 pc	INVALID_INPUT	Absolute magnitude: the distance must be a positive number.
Absolute magnitude: m = 6.5	NEEDS_INPUT	absolute_magnitude needs d (distance (pc, ly, m ...)). Example: solve('absolute_magnitude', m=12, d='3.08568e+19 m').

Flux ratio from a magnitude difference — `flux_ratio_from_magnitudes`

How many times brighter star 1 is than star 2: F1/F2 = 10^(-0.4 $(m_1 - m_2)$).

$$\frac{F_1}{F_2} = 10^{-0.4\,(m_1 - m_2)}$$

input	meaning	condition
`m1`	magnitude of the first object	
`m2`	magnitude of the second object	

output	meaning
`ratio`	flux ratio F_1/F_2
`dm`	magnitude difference $m_1 - m_2$

- **Method:** Pogson's scale: F1/F2 = 10^(-0.4 $(m_1 - m_2)$)
- **Assumes:** $|m_1 - m_2|$ <= 100
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('flux_ratio_from_magnitudes', m1=1.0, m2=6.0)
# -> ratio = 100, dm = -5   (SOLVED)
```

```
astronomy_ai999x.solve('flux_ratio_from_magnitudes', m1=0.0, m2=2.5)
# -> ratio = 10, dm = -2.5   (SOLVED)
```

```
astronomy_ai999x.solve('flux_ratio_from_magnitudes', m1=4.0, m2=4.0)
# -> ratio = 1, dm = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Flux ratio: m1 = 1, m2 = 6	SOLVED	ratio = 100, dm = -5
Flux ratio: m1 = 0 mag, m2 = 5 mag	SOLVED	ratio = 100, dm = -5
Flux ratio: m1 = 0, m2 = 500	INVALID_INPUT	Flux ratio from a magnitude difference: the magnitude difference must be at most 100.
Flux ratio: m1 = 1	NEEDS_INPUT	flux_ratio_from_magnitudes needs m2 (magnitude of the second object). Example: solve('flux_ratio_from_magnitudes', m1=1,

Magnitude difference from a flux ratio — `magnitude_difference_from_flux_ratio`

The magnitude difference $m_1 - m_2$ of two objects whose fluxes are in the ratio F_1/F_2.

$$m_1 - m_2 = -2.5 \log_{10} \frac{F_1}{F_2}$$

input	meaning	condition
`ratio`	flux ratio F_1/F_2	

output	meaning
`dm`	magnitude difference $m_1 - m_2$

- **Method:** $m_1 - m_2 = -2.5$ log10(F1/F2)
- **Assumes:** $F_1/F_2 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('magnitude_difference_from_flux_ratio', ratio=100.0)
# -> dm = -5   (SOLVED)
```

```
astronomy_ai999x.solve('magnitude_difference_from_flux_ratio', ratio=0.01)
# -> dm = 5   (SOLVED)
```

```
astronomy_ai999x.solve('magnitude_difference_from_flux_ratio', ratio=10.0)
# -> dm = -2.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Magnitude difference: ratio = 100	SOLVED	dm = -5

question	outcome	result
Magnitude difference from flux ratio: ratio = 0.01	SOLVED	dm = 5
Magnitude difference: ratio = -2	INVALID_INPUT	Magnitude difference from a flux ratio: the flux ratio must be a positive number.
Magnitude difference:	NEEDS_INPUT	magnitude_difference_from_flux_ratio needs ratio (flux ratio F1/F2). Example: solve('magnitude_difference_from_flux_rati

Combined magnitude of two stars — `combined_magnitude`

The magnitude of two unresolved stars seen together (their fluxes add).

$$m = -2.5 \log_{10}\left(10^{-0.4 m_1} + 10^{-0.4 m_2}\right)$$

input	meaning	condition
`m1`	magnitude of the first star	
`m2`	magnitude of the second star	

output	meaning
`m`	combined magnitude
`brightening`	how much brighter than the brighter star alone (mag)

- **Method:** fluxes add: $m = -2.5$ log10(10^(-0.4 m_1) + $10^($ -0.4 m2))
- **Assumes:** $|m_1 - m_2|$ <= 100
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('combined_magnitude', m1=5.0, m2=5.0)
# -> m = 4.24743, brightening = 0.752575   (SOLVED)
```

```
astronomy_ai999x.solve('combined_magnitude', m1=1.0, m2=6.0)
# -> m = 0.989197, brightening = 0.0108034   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Combined magnitude: m1 = 5, m2 = 5	SOLVED	m = 4.24743, brightening = 0.752575
Combined magnitude: m1 = 1, m2 = 6	SOLVED	m = 0.989197, brightening = 0.0108034
Combined magnitude: m1 = 1, m2 = 600	INVALID_INPUT	Combined magnitude of two stars: the two magnitudes must differ by at most 100.
Combined magnitude: m1 = 1	NEEDS_INPUT	combined_magnitude needs m2 (magnitude of the second star). Example: solve('combined_magnitude', m1=5, m2=5).

Absolute bolometric magnitude from luminosity — `bolometric_magnitude_from_luminosity`

The absolute bolometric magnitude of a star of luminosity L on the IAU 2015 scale (zero point $L_0 = 3.0128e28$ W).

$$M_{\mathrm{bol}} = -2.5 \log_{10} \frac{L}{L_0}$$

input	meaning	condition
`L`	luminosity (W or Lsun)	

output	meaning
`M_bol`	absolute bolometric magnitude
`L_Lsun`	luminosity in solar luminosities

- **Method:** M_bol = -2.5 log10(L / L0), $L_0 = 3.0128e28$ W (IAU 2015 B2)
- **Assumes:** $L > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('bolometric_magnitude_from_luminosity', L=3.0128e+28)
# -> M_bol = 0, L_Lsun = 78.7043   (SOLVED)
```

```
astronomy_ai999x.solve('bolometric_magnitude_from_luminosity', L=3.0127999999999997e+30)
# -> M_bol = -5, L_Lsun = 7870.43   (SOLVED)
```

```
astronomy_ai999x.solve('bolometric_magnitude_from_luminosity', L=3.828e+26)
# -> M_bol = 4.74, L_Lsun = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Absolute bolometric magnitude: L = 1 Lsun	SOLVED	M_bol = 4.74, L_Lsun = 1
Absolute bolometric magnitude: L = 3.0128e30 W	SOLVED	M_bol = -5, L_Lsun = 7870.43
Absolute bolometric magnitude: L = 0 W	INVALID_INPUT	Absolute bolometric magnitude from luminosity: the luminosity must be a positive number.
Absolute bolometric magnitude:	NEEDS_INPUT	bolometric_magnitude_from_luminosity needs L (luminosity (W or Lsun)). Example: solve('bolometric_magnitude_from_luminos

Luminosity from absolute bolometric magnitude — `luminosity_from_bolometric_magnitude`

The luminosity of a star from its absolute bolometric magnitude on the IAU 2015 scale.

$$L = L_0 \, 10^{-0.4 M_{\mathrm{bol}}}$$

input	meaning	condition
`M_bol`	absolute bolometric magnitude	

output	meaning
`L`	luminosity
`L_Lsun`	luminosity in solar luminosities

- **Method:** L = L0 * 10^(-0.4 M_bol), $L_0 = 3.0128e28$ W
- **Assumes:** -30 < M_bol < 40
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('luminosity_from_bolometric_magnitude', M_bol=0.0)
# -> L = 3.0128e+28 W, L_Lsun = 78.7043   (SOLVED)
```

```
astronomy_ai999x.solve('luminosity_from_bolometric_magnitude', M_bol=-5.0)
# -> L = 3.0128e+30 W, L_Lsun = 7870.43   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Luminosity from bolometric magnitude: M_bol = -5	SOLVED	L = 3.0128e+30 W, L_Lsun = 7870.43
Luminosity from bolometric magnitude: M_bol = 0 mag	SOLVED	L = 3.0128e+28 W, L_Lsun = 78.7043
Luminosity from bolometric magnitude: M_bol = 99	INVALID_INPUT	Luminosity from absolute bolometric magnitude: the absolute bolometric magnitude must lie between -30 and 40.
Luminosity from bolometric magnitude:	NEEDS_INPUT	luminosity_from_bolometric_magnitude needs M_bol (absolute bolometric magnitude). Example: solve('luminosity_from_bolome

Apparent bolometric magnitude and flux — `apparent_bolometric_magnitude`

The bolometric flux at the observer and the apparent bolometric magnitude of a star of luminosity L at distance d.

$$F = \frac{L}{4\pi d^2}, \quad m_{\mathrm{bol}} = -2.5 \log_{10} \frac{F}{F_0}$$

input	meaning	condition
`L`	luminosity (W or Lsun)	
`d`	distance (pc, ly, m ...)	

output	meaning
`m_bol`	apparent bolometric magnitude
`F`	bolometric flux at the observer
`M_bol`	absolute bolometric magnitude

- **Method:** F = L/(4 pi d^2), m_bol = -2.5 log10(F/F0), F0 = L0/(4 pi (10 pc)^2)
- **Assumes:** $L > 0$; $d > 0$; no extinction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('apparent_bolometric_magnitude', L=3.0128e+28, d=3.0856775814913674e+18)
# -> m_bol = 5, F = 2.51802e-10 W/m^2, M_bol = 0   (SOLVED)
```

```
astronomy_ai999x.solve('apparent_bolometric_magnitude', L=3.0127999999999997e+30,
d=3.085677581491367e+19)
# -> m_bol = 5, F = 2.51802e-10 W/m^2, M_bol = -5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apparent bolometric magnitude: L = 3.0128e28 W, d = 100 pc	SOLVED	m_bol = 5, F = 2.51802e-10 W/m^2, M_bol = 0
Apparent bolometric magnitude: L = 3.0128e28 W, d = 1 kpc	SOLVED	m_bol = 10, F = 2.51802e-12 W/m^2, M_bol = 0
Apparent bolometric magnitude: L = 1 Lsun, d = 0 pc	INVALID_INPUT	Apparent bolometric magnitude and flux: the luminosity and the distance must be a positive number.
Apparent bolometric magnitude: L = 1 Lsun	NEEDS_INPUT	apparent_bolometric_magnitude needs d (distance (pc, ly, m ...)). Example: solve('apparent_bolometric_magnitude', L='3.0

Angular size — `angular_size`

The angle subtended by an object of diameter D at distance d (exact: 2 atan(D / 2d)).

$$\theta = 2 \arctan \frac{D}{2d} \approx \frac{D}{d}$$

input	meaning	condition
`D`	diameter of the object	
`d`	distance to the object	

output	meaning
`theta`	angular diameter
`theta_arcsec`	angular diameter in arcseconds
`theta_deg`	angular diameter in degrees

- **Method:** $\theta = 2$ atan(D / (2 d)); for D << d this is D/d radians
- **Assumes:** $D > 0$; $d > 0$ (distance to the object's centre plane)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('angular_size', D=2.0, d=1.0)
# -> theta = 1.5708 rad, theta_arcsec = 324000, theta_deg = 90   (SOLVED)
```

```
astronomy_ai999x.solve('angular_size', D=3.4641016151377544, d=1.0)
# -> theta = 2.0944 rad, theta_arcsec = 432000, theta_deg = 120   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Angular size: D = 2 m, d = 1 m	SOLVED	theta = 1.5708 rad, theta_arcsec = 324000, theta_deg = 90
Angular size: D = 3474.8 km, d = 384400 km	SOLVED	theta = 0.00903948 rad, theta_arcsec = 1864.53, theta_deg = 0.517924
Angular size: D = 2 m, d = -1 m	INVALID_INPUT	Angular size: the size and the distance must be a positive number.
Angular size: D = 2 m	NEEDS_INPUT	angular_size needs d (distance to the object). Example: solve('angular_size', D='2 m', d='1 m').

Linear size from angular size — `linear_size_from_angle`

The true diameter of an object of angular size theta at distance d (exact: 2 d tan(theta/2)).

$$D = 2d\tan\frac{\theta}{2} \approx \theta d$$

input	meaning	condition
`theta`	angular diameter (arcsec, arcmin, deg, rad)	
`d`	distance to the object	

output	meaning
`D`	diameter
`D_km`	diameter in kilometres
`D_au`	diameter in astronomical units

- **Method:** $D = 2$ d tan(theta/2); for small angles $D = \theta$ d
- **Assumes:** $0 < \theta < 180$ degrees; $d > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('linear_size_from_angle', theta=1.5707963267948966, d=1.0)
# -> D = 2 m, D_km = 0.002, D_au = 1.33692e-11   (SOLVED)
```

```
astronomy_ai999x.solve('linear_size_from_angle', theta=2.0943951023931953, d=1.0)
# -> D = 3.4641 m, D_km = 0.0034641, D_au = 2.31561e-11   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Linear size: theta = 90 deg, d = 1 m	SOLVED	D = 2 m, D_km = 0.002, D_au = 1.33692e-11
Linear size: theta = 1 arcsec, d = 1 pc	SOLVED	D = 1.49598e+11 m, D_km = 1.49598e+08, D_au = 1
Linear size: theta = 200 deg, d = 1 m	INVALID_INPUT	Linear size from angular size: the angular size must be smaller than 180 degrees.
Linear size: theta = 1 arcsec	NEEDS_INPUT	linear_size_from_angle needs d (distance to the object). Example: solve('linear_size_from_angle', theta='1.5708 rad', d=

Luminosity of a star (Stefan-Boltzmann) — `stellar_luminosity`

The luminosity of a star of radius R radiating as a blackbody at effective temperature T: $L = 4$ pi R^2 sigma T^4.

$$L = 4\pi R^2 \sigma T_{\text{eff}}^4$$

input	meaning	condition
`R`	stellar radius (m, km, Rsun)	
`T`	effective temperature	

output	meaning
`L`	luminosity
`L_Lsun`	luminosity in solar luminosities

- **Method:** $L = 4$ pi R^2 sigma T^4 with $\sigma = 2\,pi^5$ k^4 / (15 h^3 c^2)
- **Assumes:** $R > 0$; $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('stellar_luminosity', R=1.0, T=100.0)
# -> L = 71.256 W, L_Lsun = 1.86144e-25   (SOLVED)
```

```
astronomy_ai999x.solve('stellar_luminosity', R=695700000.0, T=5772.0)
# -> L = 3.82799e+26 W, L_Lsun = 0.999998   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stellar luminosity: R = 1 Rsun, T = 5772 K	SOLVED	L = 3.82799e+26 W, L_Lsun = 0.999998
Stellar luminosity: R = 1 m, T = 100 K	SOLVED	L = 71.256 W, L_Lsun = 1.86144e-25
Stellar luminosity: R = 1 Rsun, T = -5 K	INVALID_INPUT	Luminosity of a star (Stefan-Boltzmann): the radius and the temperature must be a positive number.
Stellar luminosity: R = 1 Rsun	NEEDS_INPUT	stellar_luminosity needs T (effective temperature). Example: solve('stellar_luminosity', R='1 m', T='100 K').

Radius of a star from luminosity and temperature — `stellar_radius`

The radius of a star from its luminosity and effective temperature: $R = sqrt$(L / (4 pi sigma T^4)).

$$R = \sqrt{\frac{L}{4\pi\sigma T_{\text{eff}}^4}}$$

input	meaning	condition
`L`	luminosity (W or Lsun)	
`T`	effective temperature	

output	meaning
`R`	stellar radius
`R_Rsun`	radius in solar radii

- **Method:** $R = sqrt$(L / (4 pi sigma)) / T^2
- **Assumes:** $L > 0$; $T > 0$; blackbody emission
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('stellar_radius', L=71.25602647365321, T=100.0)
# -> R = 1 m, R_Rsun = 1.4374e-09   (SOLVED)
```

```
astronomy_ai999x.solve('stellar_radius', L=28502410589.461285, T=100.0)
# -> R = 20000 m, R_Rsun = 2.8748e-05   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stellar radius: L = 1 Lsun, T = 5772 K	SOLVED	R = 6.95701e+08 m, R_Rsun = 1
Stellar radius: L = 1 Lsun, T = 0 K	INVALID_INPUT	Radius of a star from luminosity and temperature: the luminosity and the temperature must be a positive number.
Stellar radius: T = 5772 K	NEEDS_INPUT	stellar_radius needs L (luminosity (W or Lsun)). Example: solve('stellar_radius', L='71.256 W', T='100 K').

Effective temperature of a star — `effective_temperature`

The effective temperature of a star from its luminosity and radius: T = (L / (4 pi R^2 sigma))^$(1/4)$.

$$T_{\text{eff}} = \left(\frac{L}{4\pi R^2 \sigma}\right)^{1/4}$$

input	meaning	condition
`L`	luminosity (W or Lsun)	
`R`	stellar radius (m, km, Rsun)	

output	meaning
`T`	effective temperature

- **Method:** T = (L / (4 pi R^2 sigma))^$(1/4)$
- **Assumes:** $L > 0$; $R > 0$; blackbody emission
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('effective_temperature', L=71.25602647365321, R=1.0)
# -> T = 100 K   (SOLVED)
```

```
astronomy_ai999x.solve('effective_temperature', L=4453501654603.325, R=4.0)
# -> T = 25000 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Effective temperature: L = 1 Lsun, R = 1 Rsun	SOLVED	T = 5772 K
Effective temperature: L = 1 Lsun, R = 0 m	INVALID_INPUT	Effective temperature of a star: the luminosity and the radius must be a positive number.
Effective temperature: L = 1 Lsun	NEEDS_INPUT	effective_temperature needs R (stellar radius (m, km, Rsun)). Example: solve('effective_temperature', L='71.256 W', R='1

Equilibrium temperature of a planet — `planet_equilibrium_temperature`

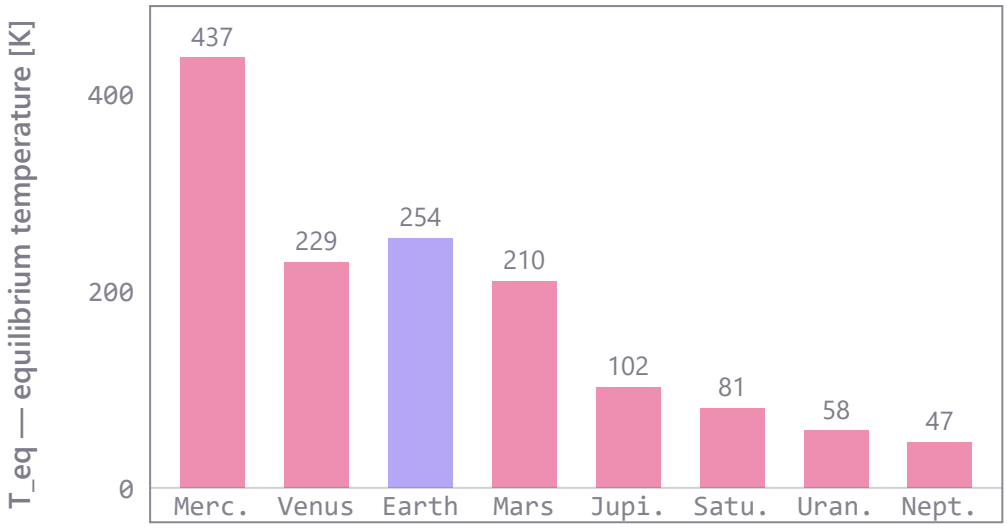
The equilibrium temperature of a rapidly rotating planet of Bond albedo A at distance a from a star of temperature T and radius R, and the stellar flux it receives.

$$T_{\mathrm{eq}} = T_\star \sqrt{\frac{R_\star}{2a}}\,(1-A)^{1/4}$$

input	meaning	condition
`T_star`	effective temperature of the star	
`R_star`	radius of the star (m, Rsun)	
`a`	orbital distance (m, au)	
`A`	Bond albedo (0 to 1, default 0)	

output	meaning
`T_eq`	equilibrium temperature
`S`	stellar flux at the planet

- **Method:** $(1-A)$ S pi $Rp^2 = 4$ pi Rp^2 sigma T^4 with $S = \sigma$ T*^4 (R*/a)^2: $T = T sqrt$(R*/(2a)) $(1-A)^{1/4}$
- **Assumes:** $0 \le A < 1$; a > R_star; full heat redistribution, unit emissivity
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Equilibrium temperatures of the eight planets for the Sun (5772 K, radius 6.957e8 m), from their mean distances and Bond albedos (inputs), each computed by the package: Mercury 437.223 K, Venus 229.11 K, Earth 254.038 K, Mars 209.813 K, Jupiter 102.452 K, Saturn 81.1727 K, Uranus 58.1161 K, Neptune 46.5916 K (Earth highlighted; these are no-atmosphere values, without any greenhouse warming).

```
astronomy_ai999x.solve('planet_equilibrium_temperature', T_star=1000.0, R_star=2.0, a=4.0)
# -> T_eq = 500 K, S = 14175.9 W/m^2   (SOLVED)
```

```
astronomy_ai999x.solve('planet_equilibrium_temperature', T_star=1000.0, R_star=2.0, a=4.0,
A=0.9375)
# -> T_eq = 250 K, S = 14175.9 W/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Equilibrium temperature: T_star = 1000 K, R_star = 2 m, a = 4 m	SOLVED	T_eq = 500 K, S = 14175.9 W/m^2
Equilibrium temperature: T_star = 5772 K, R_star = 1 Rsun, a = 1 au, A = 0.3	SOLVED	T_eq = 254.586 K, S = 1361.16 W/m^2
Equilibrium temperature: T_star = 5772 K, R_star = 1 Rsun, a = 1 au, A = 1.5	INVALID_INPUT	Equilibrium temperature of a planet: the Bond albedo A must satisfy 0 <= A < 1.
Equilibrium temperature: T_star = 5772 K, a = 1 au	NEEDS_INPUT	planet_equilibrium_temperature needs R_star (radius of the star (m, Rsun)). Example: solve('planet_equilibrium_temperatu

Ionization fraction of hydrogen (Saha equation) — `saha_hydrogen_ionization`

The ionized fraction of pure hydrogen in thermal equilibrium at temperature T and total hydrogen density n (Saha equation, ground-state partition functions).

$$\frac{x^2}{1-x} = \frac{1}{n}\left(\frac{2\pi m_e kT}{h^2}\right)^{3/2} e^{-\chi/kT}$$

input	meaning	condition
`T`	temperature	
`n`	total density of hydrogen nuclei (m^-3 or cm^-3)	

output	meaning
`x`	ionized fraction n_II / n
`neutral`	neutral fraction n_I / n
`n_e`	electron density

- **Method:** $x^2/(1-x) = (1/n)$ (2 pi m_e k T / h^2)^(3/2) exp(-chi / kT), solved in a cancellation-free form
- **Assumes:** 1000 $K \leq T \leq 1e6$ K; pure hydrogen, $n_e = n_I I$; $g_I = 2$, $g_I I = 1$, $\chi = 13.598$ eV
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('saha_hydrogen_ionization', T=10000.0, n=6.769876143152214e+20)
# -> x = 0.5, neutral = 0.5, n_e = 3.38494e+20 1/m^3   (SOLVED)
```

```
astronomy_ai999x.solve('saha_hydrogen_ionization', T=8000.0, n=2.343302528357021e+18)
# -> x = 0.732051, neutral = 0.267949, n_e = 1.71542e+18 1/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Saha equation: T = 10000 K, n = 1e20 m^-3	SOLVED	x = 0.807409, neutral = 0.192591, n_e = 8.07409e+19 1/m^3
Saha equation: T = 300 K, n = 1e20 m^-3	INVALID_INPUT	Ionization fraction of hydrogen (Saha equation): the temperature must lie between 1000 K and 1e6 K.
Saha equation: T = 10000 K	NEEDS_INPUT	saha_hydrogen_ionization needs n (total density of hydrogen nuclei (m^-3 or cm^-3)). Example: solve('saha_hydrogen_ioniz

Schwarzschild radius — `schwarzschild_radius`

The Schwarzschild radius (event horizon of a non-rotating black hole) of a mass M: $r_s = 2\,\mathrm{G}\,M/c^2$.

$$r_s = \frac{2GM}{c^2}$$

input	meaning	condition
`M`	mass (kg, Msun, Mearth)	

output	meaning
`rs`	Schwarzschild radius

output	meaning
`rs_km`	Schwarzschild radius in kilometres
`M_Msun`	mass in solar masses

- **Method:** $r_s = 2$ G M/c^2
- **Assumes:** $M > 0$; non-rotating, uncharged mass
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('schwarzschild_radius', M=1.988409870698051e+30)
# -> rs = 2953.25 m, rs_km = 2.95325, M_Msun = 1   (SOLVED)
```

```
astronomy_ai999x.solve('schwarzschild_radius', M=6.732954607500545e+26)
# -> rs = 1 m, rs_km = 0.001, M_Msun = 0.00033861   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Schwarzschild radius: M = 1 Msun	SOLVED	rs = 2953.25 m, rs_km = 2.95325, M_Msun = 1
Schwarzschild radius: M = 5.972e24 kg	SOLVED	rs = 0.00886981 m, rs_km = 8.86981e-06, M_Msun = 3.0034e-06
Schwarzschild radius: M = 0 kg	INVALID_INPUT	Schwarzschild radius: the mass must be a positive number.
Schwarzschild radius:	NEEDS_INPUT	schwarzschild_radius needs M (mass (kg, Msun, Mearth)). Example: solve('schwarzschild_radius', M='1.98841e+30 kg').

Scale height of an isothermal atmosphere — `atmospheric_scale_height`

The pressure scale height $H = R$ T / (mu g) of an isothermal planetary atmosphere: pressure falls by a factor e every H.

$$H = \frac{RT}{\mu g}$$

input	meaning	condition
`T`	temperature of the atmosphere	
`mu`	mean molar mass (kg/mol or g/mol)	
`g`	surface gravity	

output	meaning
`H`	scale height
`H_km`	scale height in kilometres

- **Method:** $H = R$ T / (mu g)
- **Assumes:** T, mu, $g > 0$; isothermal ideal gas, constant gravity

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('atmospheric_scale_height', T=300.0, mu=0.25452436586183386, g=9.8)
# -> H = 1000 m, H_km = 1   (SOLVED)
```

```
astronomy_ai999x.solve('atmospheric_scale_height', T=288.0, mu=0.02897, g=9.81)
# -> H = 8425.76 m, H_km = 8.42576   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Scale height: T = 288 K, mu = 28.97 g/mol, g = 9.81 m/s^2	SOLVED	H = 8425.76 m, H_km = 8.42576
Scale height: T = 288 K, mu = 0 g/mol, g = 9.81 m/s^2	INVALID_INPUT	Scale height of an isothermal atmosphere: the temperature, the molar mass and the gravity must be a positive number.
Scale height: T = 288 K, g = 9.81 m/s^2	NEEDS_INPUT	atmospheric_scale_height needs mu (mean molar mass (kg/mol or g/mol)). Example: solve('atmospheric_scale_height', T='300

Free-fall time of a uniform cloud — `free_fall_time`

The time a uniform, pressureless sphere of density rho takes to collapse from rest under its own gravity.

$$t_{\mathrm{ff}} = \sqrt{\frac{3\pi}{32\,G\rho}}$$

input	meaning	condition
`rho`	mean density (kg/m^3; g/cm^3)	

output	meaning
`t_ff`	free-fall time
`t_ff_yr`	free-fall time in years

- **Method:** $t_f f = sqrt$(3 pi / (32 G rho))
- **Assumes:** $\rho > 0$; uniform, pressureless, non-rotating sphere starting at rest
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('free_fall_time', rho=4412811999.3713665)
# -> t_ff = 1 s, t_ff_yr = 3.16881e-08   (SOLVED)
```

```
astronomy_ai999x.solve('free_fall_time', rho=1103202999.8428416)
# -> t_ff = 2 s, t_ff_yr = 6.33762e-08   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Free-fall time: rho = 1e-17 kg/m^3	SOLVED	t_ff = 2.10067e+13 s, t_ff_yr = 665662
Free-fall time: rho = -1 kg/m^3	INVALID_INPUT	Free-fall time of a uniform cloud: the density must be a positive number.
Free-fall time:	NEEDS_INPUT	free_fall_time needs rho (mean density (kg/m^3; g/cm^3)). Example: solve('free_fall_time', rho='4.41281e+09 kg/m^3').

Masses of a visual binary star — `binary_star_masses`

The total and individual masses of a binary star from its orbital period and each star's semi-major axis about the centre of mass (Kepler's third law and the centre-of-mass condition).

$$M_1 + M_2 = \frac{4\pi^2 (a_1 + a_2)^3}{G P^2}, \quad M_1 a_1 = M_2 a_2$$

input	meaning	condition
`P`	orbital period (s, day, yr)	
`a1`	semi-major axis of star 1 about the centre of mass (au, m)	
`a2`	semi-major axis of star 2 about the centre of mass (au, m)	

output	meaning
`M_total`	total mass
`M_total_Msun`	total mass in solar masses
`M1_Msun`	mass of star 1 in solar masses
`M2_Msun`	mass of star 2 in solar masses
`a_au`	relative semi-major axis $a_1 + a_2$ in au

- **Method:** $M_1 + M_2 = 4\,pi^2$ (a1 + a2)^3 / (G P^2); M1 $a_1 = M_2$ a2
- **Assumes:** P, a1, $a_2 > 0$; true (de-projected) semi-major axes
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('binary_star_masses', P=6.283185307179586, a1=0.25, a2=0.75)
# -> M_total = 1.49828e+10 kg, M_total_Msun = 7.53509e-21, M1_Msun = 5.65132e-21, M2_Msun =
1.88377e-21, a_au = 6.68459e-12   (SOLVED)
```

```
astronomy_ai999x.solve('binary_star_masses', P=31557600.0, a1=74798935350.0, a2=74798935350.0)
# -> M_total = 1.98848e+30 kg, M_total_Msun = 1.00004, M1_Msun = 0.500019, M2_Msun = 0.500019, a_au
= 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Binary star masses: P = 1 yr, a1 = 0.25 au, a2 = 0.75 au	SOLVED	M_total = 1.98848e+30 kg, M_total_Msun = 1.00004, M1_Msun = 0.750028, M2_Msun = 0.250009, a_au = 1

question	outcome	result
Binary star masses: P = 1 yr, a1 = 0 au, a2 = 0.75 au	INVALID_INPUT	Masses of a visual binary star: the period and the two semi-major axes must be a positive number.
Binary star masses: P = 1 yr, a1 = 0.25 au	NEEDS_INPUT	binary_star_masses needs a2 (semi-major axis of star 2 about the centre of mass (au, m)). Example: solve('binary_star_ma

Hohmann transfer between circular orbits — `hohmann_transfer`

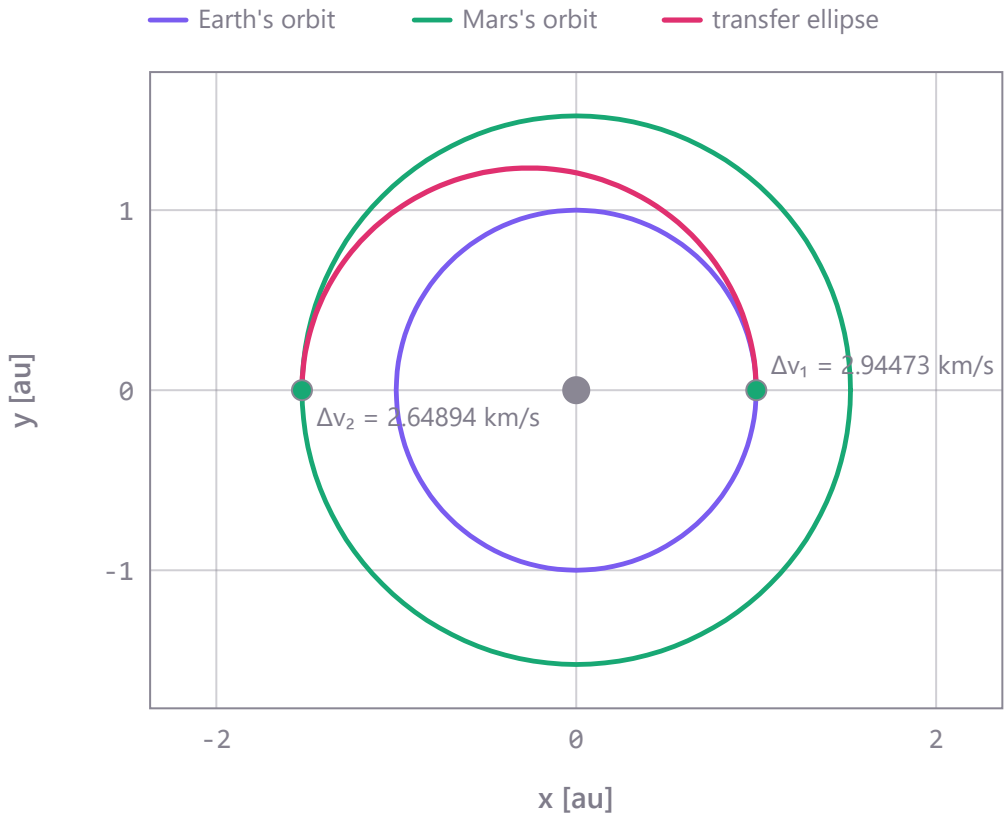
The two velocity changes, their sum and the transfer time of a Hohmann transfer between coplanar circular orbits of radii r1 and r2 around a body of mass M.

$$\Delta v_1 = \sqrt{\frac{\mu}{r_1}}\left(\sqrt{\frac{2r_2}{r_1+r_2}}-1\right),\ \Delta v_2 = \sqrt{\frac{\mu}{r_2}}\left(1-\sqrt{\frac{2r_1}{r_1+r_2}}\right),\ t=\pi\sqrt{\frac{a^3}{\mu}}$$

input	meaning	condition
`r1`	radius of the initial circular orbit (km, au)	
`r2`	radius of the final circular orbit	
`M`	mass of the central body (kg, Mearth, Msun)	

output	meaning
`dv1`	first burn (departure)
`dv2`	second burn (arrival)
`dv`	total delta-v
`t_transfer`	transfer time (half the transfer ellipse)
`t_days`	transfer time in days
`a_transfer`	semi-major axis of the transfer ellipse

- **Method:** $vp = sqrt$(2 mu r2 / (r1 $(r_1 + r_2)$)), $va = sqrt$(2 mu r1 / (r2 $(r_1 + r_2)$)); $t = pi$ sqrt(a^3/mu)
- **Assumes:** r1, r2, $M > 0$; $r_1 \neq r_2$; impulsive burns, coplanar circular orbits
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Hohmann transfer from a circular orbit of r_1 = 1 au to one of r_2 = 1.52368 au around the Sun, computed by the package: transfer ellipse a = 1.26184 au, e = 0.207506 (drawn from 91 positions it solved over the 258.862-day half orbit), burns Δv_1 = 2.94473 km/s and Δv_2 = 2.64894 km/s, total 5.59367 km/s.

```
astronomy_ai999x.solve('hohmann_transfer', r1=1.0, r2=3.0, M=14982844642.8839)
# -> dv1 = 0.224745 m/s, dv2 = 0.169102 m/s, dv = 0.393847 m/s, t_transfer = 8.88577 s, t_days =
0.000102845, a_transfer = 2 m   (SOLVED)
```

```
astronomy_ai999x.solve('hohmann_transfer', r1=3.0, r2=1.0, M=14982844642.8839)
# -> dv1 = 0.169102 m/s, dv2 = 0.224745 m/s, dv = 0.393847 m/s, t_transfer = 8.88577 s, t_days =
0.000102845, a_transfer = 2 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hohmann transfer: r1 = 6678 km, r2 = 42164 km, M = 5.972e24 kg	SOLVED	dv1 = 2425.73 m/s, dv2 = 1466.82 m/s, dv = 3892.55 m/s, t_transfer = 18990.3 s, t_days = 0.219795, a_transfer = 2.4421e+07 m
Hohmann transfer: r1 = 7000 km, r2 = 7000 km, M = 1 Mearth	INVALID_INPUT	Hohmann transfer between circular orbits: the two orbits must have different radii.
Hohmann transfer: r1 = 7000 km, r2 = 42164 km	NEEDS_INPUT	hohmann_transfer needs M (mass of the central body (kg, Mearth, Msun)). Example: solve('hohmann_transfer', r1='1 m', r2=

Speeds at periapsis and apoapsis — `apsis_speeds`

The orbital speeds at the closest and farthest points of an elliptical orbit, with its semi-major axis, eccentricity and period.

$$v_p = \sqrt{\frac{2\mu\, r_a}{r_p(r_p + r_a)}}, \quad v_a = \sqrt{\frac{2\mu\, r_p}{r_a(r_p + r_a)}}$$

input	meaning	condition
`rp`	periapsis distance from the centre of the central body	
`ra`	apoapsis distance	
`M`	mass of the central body	

output	meaning
`vp`	speed at periapsis
`va`	speed at apoapsis
`a`	semi-major axis
`e`	eccentricity
`P`	orbital period

- **Method:** energy and angular momentum: $vp = sqrt$(2 mu ra / (rp (rp + ra))), $va = vp$ rp / ra
- **Assumes:** $0 < rp < ra$; $M > 0$; two-body orbit, the orbiting mass negligible
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('apsis_speeds', rp=1.0, ra=3.0, M=14982844642.8839)
# -> vp = 1.22474 m/s, va = 0.408248 m/s, a = 2 m, e = 0.5, P = 17.7715 s   (SOLVED)
```

```
astronomy_ai999x.solve('apsis_speeds', rp=1.0, ra=7.0, M=14982844642.8839)
# -> vp = 1.32288 m/s, va = 0.188982 m/s, a = 4 m, e = 0.75, P = 50.2655 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apsis speeds: rp = 1 m, ra = 3 m, M = 1.4982e10 kg	SOLVED	vp = 1.22471 m/s, va = 0.408237 m/s, a = 2 m, e = 0.5, P = 17.772 s
Apsis speeds: rp = 3 au, ra = 1 au, M = 1 Msun	INVALID_INPUT	Speeds at periapsis and apoapsis: the periapsis distance must not exceed the apoapsis distance.
Apsis speeds: rp = 1 au, ra = 3 au	NEEDS_INPUT	apsis_speeds needs M (mass of the central body). Example: solve('apsis_speeds', rp='1 m', ra='3 m', M='1.49828e+10 kg').

Departure burn from a circular orbit — `escape_delta_v`

The velocity change that takes a spacecraft from a circular orbit of radius r onto an escape (or hyperbolic) trajectory with excess speed v_inf.

$$\Delta v = \sqrt{v_\infty^2 + \frac{2\mu}{r}} - \sqrt{\frac{\mu}{r}}$$

input	meaning	condition
`M`	mass of the central body	
`r`	radius of the circular orbit	
`v_inf`	hyperbolic excess speed (default 0: just escape)	

output	meaning
`dv`	departure delta-v
`v_circ`	circular orbital speed
`v_depart`	speed needed after the burn

- **Method:** $dv = sqrt$(v_inf^2 + 2 mu / r) - sqrt(mu / r)
- **Assumes:** M, $r > 0$; v_inf >= 0; a single impulsive burn
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('escape_delta_v', M=14982844642.8839, r=1.0)
# -> dv = 0.414214 m/s, v_circ = 1 m/s, v_depart = 1.41421 m/s   (SOLVED)
```

```
astronomy_ai999x.solve('escape_delta_v', M=14982844642.8839, r=1.0, v_inf=1.4142135623730951)
# -> dv = 1 m/s, v_circ = 1 m/s, v_depart = 2 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Escape delta-v: M = 5.972e24 kg, r = 6678 km	SOLVED	dv = 3200.1 m/s, v_circ = 7725.73 m/s, v_depart = 10925.8 m/s
Departure burn: M = 1 Mearth, r = 6678 km, v_inf = -1 km/s	INVALID_INPUT	Departure burn from a circular orbit: the hyperbolic excess speed must not be negative.
Departure burn: M = 1 Mearth	NEEDS_INPUT	escape_delta_v needs r (radius of the circular orbit). Example: solve('escape_delta_v', M='1.49828e+10 kg', r='1 m').

Delta-v of a combined plane change — `plane_change_delta_v`

The velocity change needed to turn a velocity v1 into a velocity v2 at an angle di to it (a plane change, possibly combined with a speed change).

$$\Delta v = \sqrt{v_1^2 + v_2^2 - 2 v_1 v_2 \cos \Delta i}$$

input	meaning	condition
`v1`	speed before the burn	
`v2`	speed after the burn	
`di`	angle between the two velocities (plane-change angle)	

output	meaning
`dv`	delta-v of the burn

- **Method:** law of cosines: $dv^2 = v_1^2 + v_2^2 - 2$ v1 v2 cos(di), written as $(v_1 - v_2)^2 + 4$ v1 v2 sin^2(di/2)
- **Assumes:** v1, $v_2 > 0$; $0 \le di \le 180$ degrees; an impulsive burn
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('plane_change_delta_v', v1=1.0, v2=1.0, di=1.0471975511965976)
# -> dv = 1 m/s   (SOLVED)
```

```
astronomy_ai999x.solve('plane_change_delta_v', v1=3.0, v2=4.0, di=1.5707963267948966)
# -> dv = 5 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plane change: v1 = 3 km/s, v2 = 4 km/s, di = 90 deg	SOLVED	dv = 5000 m/s
Plane change: v1 = 7.7 km/s, v2 = 7.7 km/s, di = 60 deg	SOLVED	dv = 7700 m/s
Plane change: v1 = 3 km/s, v2 = 4 km/s, di = 200 deg	INVALID_INPUT	Delta-v of a combined plane change: the angle between the velocities must lie between 0 and 180 degrees.
Plane change: v1 = 3 km/s, di = 90 deg	NEEDS_INPUT	plane_change_delta_v needs v2 (speed after the burn). Example: solve('plane_change_delta_v', v1='1 m/s', v2='1 m/s', di=

Orbital elements from position and velocity — `orbit_from_state`

The semi-major axis, eccentricity, periapsis and apoapsis distances, period and specific energy of a bound orbit from the distance, speed and flight-path angle at one point.

$$a = \frac{r}{2 - rv^2/\mu}, \quad e = \sqrt{1 - \frac{(rv\cos\gamma)^2}{\mu a}}$$

input	meaning	condition
`M`	mass of the central body	
`r`	distance from the centre of the central body	
`v`	orbital speed at that point	
`gamma`	flight-path angle above the local horizontal (default 0)	

output	meaning
`a`	semi-major axis
`e`	eccentricity
`rp`	periapsis distance
`ra`	apoapsis distance
`P`	orbital period
`energy`	specific orbital energy (J/kg)

- **Method:** vis-viva a = r / (2 - r v^2/mu); e^2 = (s cos^2 $g - 1)^2 + (s$ cos g sin g)^2 with $s = r\,v^2/mu$

- **Assumes:** $v^2 < 2$ G M / r (bound orbit); |gamma| < 90 degrees; two-body motion
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('orbit_from_state', M=14982844642.8839, r=1.0, v=1.0)
# -> a = 1 m, e = 0, rp = 1 m, ra = 1 m, P = 6.28319 s, energy = -0.5 J/kg   (SOLVED)
```

```
astronomy_ai999x.solve('orbit_from_state', M=14982844642.8839, r=1.0, v=1.224744871391589)
# -> a = 2 m, e = 0.5, rp = 1 m, ra = 3 m, P = 17.7715 s, energy = -0.25 J/kg   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Orbit from state: M = 1.4982e10 kg, r = 1 m, v = 1.2 m/s	SOLVED	a = 1.78597 m, e = 0.440081, rp = 1 m, ra = 2.57195 m, P = 14.997 s, energy = -0.279944 J/kg
Orbit from state: M = 1 Mearth, r = 7000 km, v = 20 km/s	INVALID_INPUT	Orbital elements from position and velocity: the orbit is not bound (the speed is at or above the escape speed).
Orbit from state: M = 1 Mearth, r = 7000 km	NEEDS_INPUT	orbit_from_state needs v (orbital speed at that point). Example: solve('orbit_from_state', M='1.49828e+10 kg', r='1 m',

Position on an elliptical orbit at a given time (Kepler's equation) — `kepler_position_at_time`

The mean, eccentric and true anomalies, the distance and the speed of a body on an elliptical orbit a time t after periapsis passage.

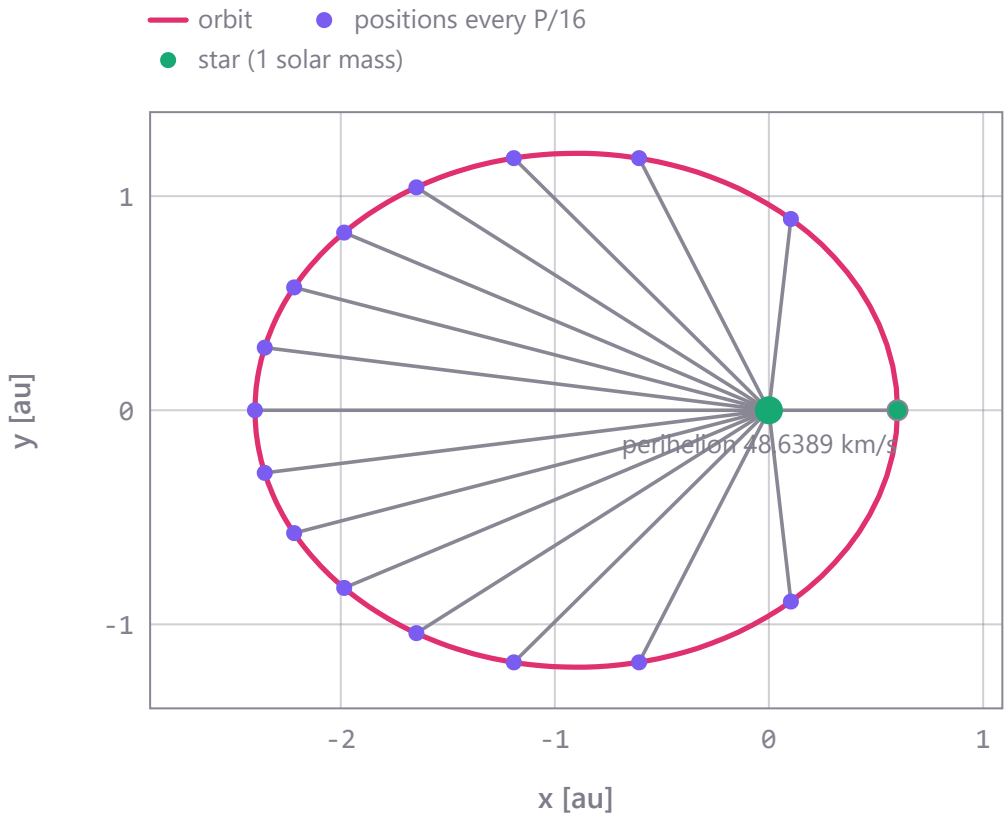
$$M = E - e\sin E, \quad \tan\frac{\nu}{2} = \sqrt{\frac{1+e}{1-e}}\tan\frac{E}{2}, \quad r = a(1 - e\cos E)$$

input	meaning	condition
`M`	mass of the central body	
`a`	semi-major axis	
`e`	eccentricity $(0 \le e < 0.99)$	
`t`	time since periapsis passage	

output	meaning
`mean_anomaly`	mean anomaly (degrees)
`E`	eccentric anomaly (degrees)
`nu`	true anomaly (degrees, 0 to 360)
`r`	distance from the central body's centre
`v`	orbital speed
`P`	orbital period

- **Method:** Kepler's equation $M = E - e$ sin E solved by safeguarded Newton iteration; tan(nu/2) = sqrt($(1+e)/(1-e)$) tan(E/2); $r = a(1 - e$ cos E)
- **Assumes:** $0 \le e < 0.99$; $t \ge 0$ (reduced modulo the period); two-body motion

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



An orbit with a = 1.5 au and e = 0.6 around one solar mass: 181 positions (true anomaly and distance) solved by the package from Kepler's equation over one period P = 671.01 days, and 16 positions at equal time steps — equal areas swept, bunched near aphelion; marked: perihelion r = 0.6 au, speed 48.6389 km/s (aphelion 12.1597 km/s).

```
astronomy_ai999x.solve('kepler_position_at_time', M=14982844642.8839, a=1.0, e=0.0,
t=1.5707963267948966)
# -> mean_anomaly = 90, E = 90, nu = 90, r = 1 m, v = 1 m/s, P = 6.28319 s   (SOLVED)
```

```
astronomy_ai999x.solve('kepler_position_at_time', M=14982844642.8839, a=1.0, e=0.5,
t=3.141592653589793)
# -> mean_anomaly = 180, E = 180, nu = 180, r = 1.5 m, v = 0.57735 m/s, P = 6.28319 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Kepler's equation: M = 1.4982e10 kg, a = 1 m, e = 0.5, t = 3.14159265358979 s	SOLVED	mean_anomaly = 179.995, E = 179.997, nu = 179.998, r = 1.5 m, v = 0.577334 m/s, P = 6.28336 s
Kepler's equation: M = 1 Msun, a = 1 au, e = 1.2, t = 100 day	INVALID_INPUT	Position on an elliptical orbit at a given time (Kepler's equation): the eccentricity must satisfy 0 <= e < 0.99 (an ell
Kepler's equation: M = 1 Msun, a = 1 au, e = 0.5	NEEDS_INPUT	kepler_position_at_time needs t (time since periapsis passage). Example: solve('kepler_position_at_time', M='1.49828e+10

Time of flight between two points of an elliptical orbit — `time_of_flight`

The time a body takes to move forward along an elliptical orbit from true anomaly nu1 to true anomaly nu2.

$$\Delta t = \frac{(E_2 - e\sin E_2) - (E_1 - e\sin E_1)}{n}, \quad n = \sqrt{\mu/a^3}$$

input	meaning	condition
`M`	mass of the central body	
`a`	semi-major axis	
`e`	eccentricity $(0 \le e < 0.99)$	
`nu1`	starting true anomaly	
`nu2`	final true anomaly	

output	meaning
`dt`	time of flight
`dt_days`	time of flight in days
`P`	orbital period

- **Method:** eccentric anomalies from the true anomalies, mean anomalies $M = E - e$ sin E, $dt = (M_2 - M_1)/n$
- **Assumes:** $0 \le e < 0.99$; $nu1 \ne nu2$; motion in the forward (prograde) direction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('time_of_flight', M=14982844642.8839, a=1.0, e=0.0, nu1=0.0,
nu2=3.141592653589793)
# -> dt = 3.14159 s, dt_days = 3.6361e-05, P = 6.28319 s   (SOLVED)
```

```
astronomy_ai999x.solve('time_of_flight', M=14982844642.8839, a=1.0, e=0.5, nu1=0.0,
nu2=3.141592653589793)
# -> dt = 3.14159 s, dt_days = 3.6361e-05, P = 6.28319 s   (SOLVED)
```

```
astronomy_ai999x.solve('time_of_flight', M=14982844642.8839, a=1.0, e=0.5, nu1=3.141592653589793,
nu2=0.0)
# -> dt = 3.14159 s, dt_days = 3.6361e-05, P = 6.28319 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Time of flight: M = 1.4982e10 kg, a = 1 m, e = 0.5, nu1 = 0 deg, nu2 = 180 deg	SOLVED	dt = 3.14168 s, dt_days = 3.63621e-05, P = 6.28336 s
Time of flight: M = 1 Msun, a = 1 au, e = 0.5, nu1 = 30 deg, nu2 = 30 deg	INVALID_INPUT	Time of flight between two points of an elliptical orbit: the two true anomalies must differ.
Time of flight: M = 1 Msun, a = 1 au, e = 0.5, nu1 = 30 deg	NEEDS_INPUT	time_of_flight needs nu2 (final true anomaly). Example: solve('time_of_flight', M='1.49828e+10 kg', a='1 m', e=0, nu1='0

Radial-velocity semi-amplitude of a star — `radial_velocity_semi_amplitude`

The semi-amplitude K of the star's line-of-sight velocity wobble caused by an orbiting planet (or companion) of mass m.

$$K = \left(\frac{2\pi G}{P}\right)^{1/3} \frac{m \sin i}{(M_\star + m)^{2/3}\sqrt{1-e^2}}$$

input	meaning	condition
`M_star`	mass of the star (Msun, kg)	
`m_p`	mass of the planet (Mjup, Mearth, kg)	
`P`	orbital period	
`e`	eccentricity (default 0)	
`i`	orbital inclination (default 90 degrees, edge-on)	

output	meaning
`K`	radial-velocity semi-amplitude of the star
`a`	relative semi-major axis
`a_au`	relative semi-major axis in au

- **Method:** K = (2 pi G / P)^(1/3) m sin i / ($(M + m)^{2/3}$ sqrt(1 - e^2))
- **Assumes:** $0 \le e < 0.99$; $0 < i \le 180$ degrees; two-body Keplerian orbit
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('radial_velocity_semi_amplitude', M_star=14967861798.241016,
m_p=14982844.6428839, P=6.283185307179586)
# -> K = 0.001 m/s, a = 1 m, a_au = 6.68459e-12   (SOLVED)
```

```
astronomy_ai999x.solve('radial_velocity_semi_amplitude', M_star=7491422321.44195,
m_p=7491422321.44195, P=6.283185307179586, e=0.6, i=0.5235987755982988)
# -> K = 0.3125 m/s, a = 1 m, a_au = 6.68459e-12   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radial velocity semi-amplitude: M_star = 1 Msun, m_p = 1 Mjup, P = 11.86 yr	SOLVED	K = 12.4598 m/s, a = 7.78242e+11 m, a_au = 5.20222
Radial velocity semi-amplitude: M_star = 1 Msun, m_p = 1 Mjup, P = 11.86 yr, e = 1.5	INVALID_INPUT	Radial-velocity semi-amplitude of a star: the eccentricity must satisfy 0 <= e < 0.99.
Radial velocity semi-amplitude: M_star = 1 Msun, m_p = 1 Mjup	NEEDS_INPUT	radial_velocity_semi_amplitude needs P (orbital period). Example: solve('radial_velocity_semi_amplitude', M_star='1.4967

Transit depth — `transit_depth`

The fractional dimming of a uniformly bright star when a planet of radius Rp crosses it at impact parameter b (b = 0 central; grazing transits included).

$$\delta = \frac{A_{\text{overlap}}}{\pi R_\star^2} = \left(\frac{R_p}{R_\star}\right)^2 \text{(full transit)}$$

input	meaning	condition
`Rp`	planet radius (Rjup, Rearth, km)	
`Rs`	stellar radius (Rsun, km)	
`b`	impact parameter in stellar radii (default 0)	

output	meaning
`depth`	fractional flux drop
`depth_ppm`	depth in parts per million
`k`	radius ratio Rp/Rs
`grazing`	whether the planet's disk crosses the stellar limb

- **Method:** overlapping area of the two disks divided by the stellar disk area; $(Rp/Rs)^2$ for a full transit
- **Assumes:** $Rp < Rs$; $0 \le b < 1 + Rp/Rs$; uniform stellar disk (no limb darkening)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('transit_depth', Rp=1.0, Rs=10.0)
# -> depth = 0.01, depth_ppm = 10000, k = 0.1, grazing = False   (SOLVED)
```

```
astronomy_ai999x.solve('transit_depth', Rp=1.0, Rs=10.0, b=0.5)
# -> depth = 0.01, depth_ppm = 10000, k = 0.1, grazing = False   (SOLVED)
```

```
astronomy_ai999x.solve('transit_depth', Rp=1.0, Rs=10.0, b=1.0)
# -> depth = 0.00489387, depth_ppm = 4893.87, k = 0.1, grazing = True   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transit depth: Rp = 1 Rjup, Rs = 1 Rsun	SOLVED	depth = 0.0105602, depth_ppm = 10560.2, k = 0.102763, grazing = False
Transit depth: Rp = 0.1 m, Rs = 1 m, b = 0.3	SOLVED	depth = 0.01, depth_ppm = 10000, k = 0.1, grazing = False
Transit depth: Rp = 1 Rjup, Rs = 1 Rsun, b = 2	INVALID_INPUT	Transit depth: no transit: the impact parameter must satisfy 0 <= b < 1 + Rp/Rs.
Transit depth: Rp = 1 Rjup	NEEDS_INPUT	transit_depth needs Rs (stellar radius (Rsun, km)). Example: solve('transit_depth', Rp='1 m', Rs='10 m').

Transit duration — `transit_duration`

The total (first to fourth contact) and full (second to third contact) durations of a transit of a planet on a circular orbit, and its impact parameter.

$$T_{14} = \frac{P}{\pi}\arcsin\left(\frac{\sqrt{(R_\star + R_p)^2 - (bR_\star)^2}}{a\sin i}\right), \quad b = \frac{a\cos i}{R_\star}$$

input	meaning	condition
`P`	orbital period	
`a`	orbit radius (au, m)	
`Rs`	stellar radius	
`Rp`	planet radius	
`i`	orbital inclination (90 degrees = edge-on)	

output	meaning
`T14`	total transit duration
`T14_h`	total duration in hours
`T23`	full-transit duration (absent for a grazing transit)
`b`	impact parameter a cos i / Rs

- **Method:** $T14 = (P/pi)$ asin(sqrt((Rs + Rp)^2 - (b Rs)^2) / (a sin i)), T23 with Rs - Rp
- **Assumes:** circular orbit; $Rp < Rs$; $a > Rs + Rp$; $0 < i \leq 90$ degrees; $b < 1 + Rp/Rs$ (the planet transits)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('transit_duration', P=6.283185307179586, a=2.0, Rs=0.9, Rp=0.1,
i=1.5707963267948966)
# -> T14 = 1.0472 s, T23 = 0.823034 s, T14_h = 0.000290888, b = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Transit duration: P = 6.283185307179586 s, a = 2 m, Rs = 0.9 m, Rp = 0.1 m, i = 90 deg	SOLVED	T14 = 1.0472 s, T23 = 0.823034 s, T14_h = 0.000290888, b = 0
Transit duration: P = 365.25 d, a = 1 au, Rs = 1 Rsun, Rp = 1 Rearth, i = 80 deg	INVALID_INPUT	Transit duration: the planet does not transit (impact parameter b >= 1 + Rp/Rs).
Transit duration: P = 365.25 d, a = 1 au, Rs = 1 Rsun, Rp = 1 Rearth	NEEDS_INPUT	transit_duration needs i (orbital inclination (90 degrees = edge-on)). Example: solve('transit_duration', P='6.28319 s',

Cosmological distances at a redshift — `cosmological_distances`

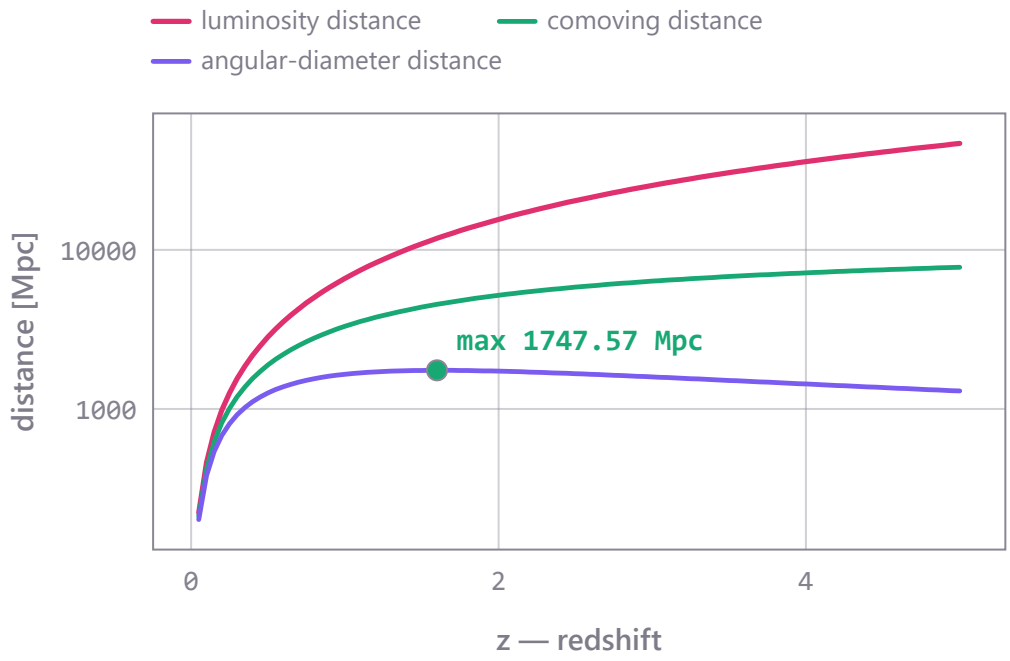
The comoving, luminosity and angular-diameter distances and the distance modulus of an object at redshift z in a flat Lambda-CDM universe.

$$D_C = \frac{c}{H_0}\int_0^z \frac{dz'}{\sqrt{\Omega_m(1+z')^3 + \Omega_\Lambda}}, \quad D_L = (1+z)D_C, \quad D_A = \frac{D_C}{1+z}$$

input	meaning	condition
`z`	redshift	
`H0`	Hubble constant in km/s/Mpc	
`Om`	matter density parameter Omega_m (flat: Omega_Lambda = 1 - Omega_m)	

output	meaning
`D_C`	comoving distance
`D_C_Mpc`	comoving distance in Mpc
`D_L_Mpc`	luminosity distance in Mpc
`D_A_Mpc`	angular-diameter distance in Mpc
`mu`	distance modulus
`D_L`	luminosity distance

- **Method:** $D_C = (c/H_0)$ integral_0^z dz'/E(z'), $E = sqrt$(Om (1+z)^3 + 1 - Om) (mpmath quadrature); $D_L = (1+z)$ D_C; $D_A = D_C/(1+z)$
- **Assumes:** $0 < z \leq 1100$; $1 \leq H_0 \leq 1000$ km/s/Mpc; $0.01 \leq Omega_m \leq 1$; flat, no radiation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Distance measures in a flat ΛCDM universe with H_0 = 70 km/s/Mpc and Ωm = 0.3, 100 redshifts each computed by the package; at z = 3: comoving 6355.69 Mpc, luminosity 25422.7 Mpc, angular-diameter 1588.92 Mpc. The angular-diameter distance peaks — marked: z = 1.6, D_A = 1747.57 Mpc (largest of the computed points).

```
astronomy_ai999x.solve('cosmological_distances', z=3.0, H0=70.0, Om=1.0)
# -> D_C = 1.32152e+26 m, D_C_Mpc = 4282.75, D_L_Mpc = 17131, D_A_Mpc = 1070.69, mu = 46.1689, D_L
= 5.28607e+26 m   (SOLVED)
```

```
astronomy_ai999x.solve('cosmological_distances', z=8.0, H0=100.0, Om=1.0)
# -> D_C = 1.23342e+26 m, D_C_Mpc = 3997.23, D_L_Mpc = 35975.1, D_A_Mpc = 444.137, mu = 47.78, D_L
= 1.11008e+27 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cosmological distances: z = 3, H0 = 70, Om = 1	SOLVED	D_C = 1.32152e+26 m, D_C_Mpc = 4282.75, D_L_Mpc = 17131, D_A_Mpc = 1070.69, mu = 46.1689, D_L = 5.28607e+26 m
Cosmological distances: z = 1, H0 = 70 km/s/Mpc, Om = 1.5	INVALID_INPUT	Cosmological distances at a redshift: Omega_m must lie between 0.01 and 1 (flat universe: Omega_Lambda = 1 - Omega_m).
Cosmological distances: z = 1, H0 = 70	NEEDS_INPUT	cosmological_distances needs Om (matter density parameter Omega_m (flat: Omega_Lambda = 1 - Omega_m)). Example: solve('c

Angular size of an object at a redshift — `angular_size_at_redshift`

The angle subtended by an object of proper size D at redshift z, and the proper length per arcsecond, in a flat Lambda-CDM universe.

$$\theta = \frac{D}{D_A}, \quad D_A = \frac{1}{1+z}\frac{c}{H_0}\int_0^z \frac{dz'}{E(z')}$$

input	meaning	condition
`z`	redshift	
`H0`	Hubble constant in km/s/Mpc	
`Om`	matter density parameter Omega_m (flat: Omega_Lambda = 1 - Omega_m)	
`D`	proper size of the object (kpc, Mpc, m)	

output	meaning
`theta_arcsec`	angular size in arcseconds
`scale_kpc_per_arcsec`	proper kiloparsecs per arcsecond
`D_A_Mpc`	angular-diameter distance in Mpc

- **Method:** $\theta = D/D_A$ with $D_A = D_C/(1+z)$
- **Assumes:** $0 < z \leq 1100$; $D > 0$; small angles; flat Lambda-CDM, no radiation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('angular_size_at_redshift', z=3.0, H0=70.0, Om=1.0,
D=1.6017254774542562e+20)
# -> theta_arcsec = 1, scale_kpc_per_arcsec = 5.19084, D_A_Mpc = 1070.69   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Angular size at redshift: z = 3, H0 = 70, Om = 1, D = 1 Mpc	SOLVED	theta_arcsec = 192.647, scale_kpc_per_arcsec = 5.19084, D_A_Mpc = 1070.69
Angular size at redshift: z = 1, H0 = 70, Om = 0.3, D = -5 kpc	INVALID_INPUT	Angular size of an object at a redshift: the size must be a positive number.
Angular size at redshift: z = 1, H0 = 70, Om = 0.3	NEEDS_INPUT	angular_size_at_redshift needs D (proper size of the object (kpc, Mpc, m)). Example: solve('angular_size_at_redshift', z

Lookback time to a redshift — `lookback_time`

How long ago the light we receive from redshift z was emitted, in a flat Lambda-CDM universe.

$$t_L = \frac{1}{H_0} \int_0^z \frac{dz'}{(1+z')\sqrt{\Omega_m (1+z')^3 + \Omega_\Lambda}}$$

input	meaning	condition
`z`	redshift	
`H0`	Hubble constant in km/s/Mpc	
`Om`	matter density parameter Omega_m (flat: Omega_Lambda = 1 - Omega_m)	

output	meaning
`t_L`	lookback time
`t_L_Gyr`	lookback time in Gyr

- **Method:** $t_L = (1/H_0)$ integral_0^z dz' / ((1 + z') E(z')) by mpmath quadrature
- **Assumes:** $0 < z \leq 1100$; flat Lambda-CDM, no radiation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('lookback_time', z=3.0, H0=70.0, Om=1.0)
# -> t_L = 2.5714e+17 s, t_L_Gyr = 8.14827   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lookback time: z = 3, H0 = 70, Om = 1	SOLVED	t_L = 2.5714e+17 s, t_L_Gyr = 8.14827
Lookback time: z = -1, H0 = 70, Om = 0.3	INVALID_INPUT	Lookback time to a redshift: the redshift must lie between 0 and 1100.
Lookback time: z = 1, Om = 0.3	NEEDS_INPUT	lookback_time needs H0 (Hubble constant in km/s/Mpc). Example: solve('lookback_time', z=3, H0=70, Om=1).

Age of the universe at a redshift — `age_at_redshift`

The age of a flat Lambda-CDM universe when the light from redshift z was emitted, and its present age.

$$t(z) = \frac{1}{H_0} \int_z^\infty \frac{dz'}{(1+z')\sqrt{\Omega_m (1+z')^3 + \Omega_\Lambda}}$$

input	meaning	condition
`z`	redshift	
`H0`	Hubble constant in km/s/Mpc	
`Om`	matter density parameter Omega_m (flat: Omega_Lambda = 1 - Omega_m)	

output	meaning
`age`	age at redshift z
`age_Gyr`	age at redshift z in Gyr
`age_now_Gyr`	present age in Gyr

- **Method:** t(z) = (1/H0) integral_z^inf dz' / ((1 + z') E(z')) by mpmath quadrature
- **Assumes:** $0 \leq z \leq 1100$; flat Lambda-CDM, no radiation (not accurate in the radiation era)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('age_at_redshift', z=3.0, H0=70.0, Om=1.0)
# -> age = 3.67343e+16 s, age_Gyr = 1.16404, age_now_Gyr = 9.31231   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Age at redshift: z = 3, H0 = 70, Om = 1	SOLVED	age = 3.67343e+16 s, age_Gyr = 1.16404, age_now_Gyr = 9.31231
Age at redshift: z = 3, H0 = 70, Om = 0	INVALID_INPUT	Age of the universe at a redshift: Omega_m must lie between 0.01 and 1 (flat universe: Omega_Lambda = 1 - Omega_m).
Age at redshift: z = 3	NEEDS_INPUT	age_at_redshift needs H0, Om (Hubble constant in km/s/Mpc; matter density parameter Omega_m (flat: Omega_Lambda = 1 - Om

Redshift at a given lookback time — `redshift_at_lookback_time`

The redshift of light emitted a lookback time t ago in a flat Lambda-CDM universe (exact inverse of the age-redshift relation).

$$1 + z = \left[\frac{\sqrt{\Omega_\Lambda / \Omega_m}}{\sinh\left(\frac{3}{2} H_0 \sqrt{\Omega_\Lambda}\,(t_0 - t_L)\right)} \right]^{2/3}$$

input	meaning	condition
`t`	lookback time (Gyr, Myr, yr)	
`H0`	Hubble constant in km/s/Mpc	
`Om`	matter density parameter Omega_m (flat: Omega_Lambda = 1 - Omega_m)	

output	meaning
`z`	redshift
`age_now_Gyr`	present age of the universe in Gyr

- **Method:** t(z) $= 2/(3$ H0 sqrt(OL)) asinh(sqrt(OL/Om) $(1+z)^{-3/2}$), inverted in closed form (Om = 1: t = 2/(3 H0) $(1+z)^{-3/2}$)
- **Assumes:** 0 < t < present age; flat Lambda-CDM, no radiation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('redshift_at_lookback_time', t=2.571397984576139e+17, H0=70.0, Om=1.0)
# -> z = 3, age_now_Gyr = 9.31231   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Redshift at lookback time: t = 8 Gyr, H0 = 70, Om = 0.3	SOLVED	z = 1.07467, age_now_Gyr = 13.467
Redshift at lookback time: t = 30 Gyr, H0 = 70, Om = 0.3	INVALID_INPUT	Redshift at a given lookback time: the lookback time must be shorter than the age of the universe.
Redshift at lookback time: t = 8 Gyr, H0 = 70	NEEDS_INPUT	redshift_at_lookback_time needs Om (matter density parameter Omega_m (flat: Omega_Lambda = 1 - Omega_m)). Example: solve

Comoving volume out to a redshift — `comoving_volume`

The all-sky comoving volume enclosed within redshift z in a flat Lambda-CDM universe.

$$V_C = \frac{4\pi}{3} D_C^3$$

input	meaning	condition
`z`	redshift	
`H0`	Hubble constant in km/s/Mpc	
`Om`	matter density parameter Omega_m (flat: Omega_Lambda = 1 - Omega_m)	

output	meaning
`V`	comoving volume
`V_Gpc3`	comoving volume in cubic gigaparsecs

- **Method:** V = (4 pi / 3) D_C^3 (flat space)
- **Assumes:** $0 < z \leq 1100$; flat Lambda-CDM, no radiation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('comoving_volume', z=3.0, H0=70.0, Om=1.0)
# -> V = 9.66737e+78 m^3, V_Gpc3 = 329.046   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Comoving volume: z = 3, H0 = 70, Om = 1	SOLVED	V = 9.66737e+78 m^3, V_Gpc3 = 329.046
Comoving volume: z = 0, H0 = 70, Om = 0.3	INVALID_INPUT	Comoving volume out to a redshift: the redshift must be positive.
Comoving volume: H0 = 70, Om = 0.3	NEEDS_INPUT	comoving_volume needs z (redshift). Example: solve('comoving_volume', z=3, H0=70, Om=1).

Radial velocity from a Doppler redshift — `recession_velocity_from_redshift`

The radial velocity of a source from its redshift interpreted as a special-relativistic Doppler shift (negative for a blueshift), with the low-velocity value cz for comparison.

$$\frac{v}{c} = \frac{(1+z)^2 - 1}{(1+z)^2 + 1}$$

input	meaning	condition
`z`	redshift (negative for a blueshift)	

output	meaning
`v`	radial velocity (positive receding)
`beta`	v/c
`v_cz`	non-relativistic estimate cz

- **Method:** v/c = ((1+z)^2 - 1) / ((1+z)^2 + 1)
- **Assumes:** $-1 < z$; a purely kinematic (special-relativistic, radial) Doppler shift; not a cosmological recession
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('recession_velocity_from_redshift', z=1.0)
# -> v = 1.79875e+08 m/s, beta = 0.6, v_cz = 2.99792e+08 m/s   (SOLVED)
```

```
astronomy_ai999x.solve('recession_velocity_from_redshift', z=2.0)
# -> v = 2.39834e+08 m/s, beta = 0.8, v_cz = 5.99585e+08 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radial velocity from redshift: z = 1	SOLVED	v = 1.79875e+08 m/s, beta = 0.6, v_cz = 2.99792e+08 m/s
Radial velocity from redshift: z = -1.5	INVALID_INPUT	Radial velocity from a Doppler redshift: the redshift must lie between -1 (excluded) and 10000.
Radial velocity from redshift:	NEEDS_INPUT	recession_velocity_from_redshift needs z (redshift (negative for a blueshift)). Example: solve('recession_velocity_from_

Redshift and radial velocity from a spectral line — `redshift_from_wavelengths`

The redshift z = lambda_obs/lambda_rest - 1 of a spectral line and the corresponding relativistic Doppler radial velocity.

$$z = \frac{\lambda_{\mathrm{obs}}}{\lambda_{\mathrm{rest}}} - 1, \quad \frac{v}{c} = \frac{(1+z)^2 - 1}{(1+z)^2 + 1}$$

input	meaning	condition
`lam_obs`	observed wavelength (nm, Angstrom, m)	
`lam_rest`	rest (laboratory) wavelength	

output	meaning
`z`	redshift
`v`	radial velocity (positive receding)
`beta`	v/c
`d_lambda`	wavelength shift

- **Method:** z = lambda_obs/lambda_rest - 1; v/c = ((1+z)^2 - 1)/((1+z)^2 + 1)
- **Assumes:** both wavelengths > 0; a purely kinematic radial Doppler shift
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('redshift_from_wavelengths', lam_obs=2e-07, lam_rest=1e-07)
# -> z = 1, v = 1.79875e+08 m/s, beta = 0.6, d_lambda = 1e-07 m   (SOLVED)
```

```
astronomy_ai999x.solve('redshift_from_wavelengths', lam_obs=3e-07, lam_rest=1e-07)
# -> z = 2, v = 2.39834e+08 m/s, beta = 0.8, d_lambda = 2e-07 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Redshift from spectral line: lam_obs = 1312.56 nm, lam_rest = 656.28 nm	SOLVED	z = 1, v = 1.79875e+08 m/s, beta = 0.6, d_lambda = 6.5628e-07 m
Redshift from spectral line: lam_obs = 0 nm, lam_rest = 656.28 nm	INVALID_INPUT	Redshift and radial velocity from a spectral line: the two wavelengths must be a positive number.
Redshift from spectral line: lam_obs = 700 nm	NEEDS_INPUT	redshift_from_wavelengths needs lam_rest (rest (laboratory) wavelength). Example: solve('redshift_from_wavelengths', lam

Diffraction limit of a telescope (Rayleigh criterion) — `rayleigh_resolution`

The smallest angular separation a circular aperture of diameter D resolves at wavelength lambda: the angle of the first dark ring of the Airy pattern (1.21967 lambda/D for small angles; often rounded to 1.22).

$$\sin\theta = \frac{j_{1,1}}{\pi}\frac{\lambda}{D} \approx 1.22\,\frac{\lambda}{D}$$

input	meaning	condition
`lam`	wavelength (nm, m)	
`D`	aperture diameter	

output	meaning
`theta`	angular resolution
`theta_arcsec`	angular resolution in arcseconds

- **Method:** sin(theta) = j_{1,1} lambda / (pi D), j_{1,1} = 3.8317059702 (first zero of J1)
- **Assumes:** lambda << D; a circular unobstructed aperture, no atmosphere
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('rayleigh_resolution', lam=0.20497349470552245, D=1.0)
# -> theta = 0.25268 rad, theta_arcsec = 52119   (SOLVED)
```

```
astronomy_ai999x.solve('rayleigh_resolution', lam=5.5e-07, D=2.4)
# -> theta = 2.79508e-07 rad, theta_arcsec = 0.0576526   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rayleigh criterion: lam = 550 nm, D = 2.4 m	SOLVED	theta = 2.79508e-07 rad, theta_arcsec = 0.0576526
Rayleigh criterion: lam = 2 m, D = 1 m	INVALID_INPUT	Diffraction limit of a telescope (Rayleigh criterion): the wavelength must be much smaller than the aperture (J11 lambda
Rayleigh criterion: lam = 550 nm	NEEDS_INPUT	rayleigh_resolution needs D (aperture diameter). Example: solve('rayleigh_resolution', lam='0.204973 m', D='1 m').

Telescope magnification, exit pupil and true field — `telescope_magnification`

The angular magnification of a telescope used with an eyepiece, the diameter of its exit pupil and (given the eyepiece's apparent field) its true field of view.

$$M = \frac{f_{\text{obj}}}{f_{\text{eye}}}, \quad d_{\text{exit}} = \frac{D}{M}$$

input	meaning	condition
`f_obj`	focal length of the objective (mm, m)	
`f_eye`	focal length of the eyepiece	
`D`	aperture diameter	
`afov`	apparent field of view of the eyepiece (optional)	

output	meaning
`M`	angular magnification
`exit_pupil`	exit-pupil diameter
`exit_pupil_mm`	exit-pupil diameter in millimetres
`tfov_deg`	true field of view in degrees (when afov is given)

- **Method:** M = f_obj / f_eye; exit pupil = D / M; true field = apparent field / M
- **Assumes:** focal lengths and aperture > 0; thin lenses, afocal (focused at infinity); true field = apparent field / M
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('telescope_magnification', f_obj=1.2, f_eye=0.025, D=0.2,
afov=0.9075712110370514)
# -> M = 48 1, exit_pupil = 0.00416667 m, exit_pupil_mm = 4.16667, tfov_deg = 1.08333   (SOLVED)
```

```
astronomy_ai999x.solve('telescope_magnification', f_obj=2.0, f_eye=0.01, D=0.2)
# -> M = 200 1, exit_pupil = 0.001 m, exit_pupil_mm = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Telescope magnification: f_obj = 1200 mm, f_eye = 25 mm, D = 200 mm, afov = 52 deg	SOLVED	M = 48 1, exit_pupil = 0.00416667 m, exit_pupil_mm = 4.16667, tfov_deg = 1.08333
Telescope magnification: f_obj = 1200 mm, f_eye = 0 mm, D = 200 mm	INVALID_INPUT	Telescope magnification, exit pupil and true field: the focal lengths and the aperture must be a positive number.
Telescope magnification: f_obj = 1200 mm, D = 200 mm	NEEDS_INPUT	telescope_magnification needs f_eye (focal length of the eyepiece). Example: solve('telescope_magnification', f_obj='1.2

Light-gathering power of two telescopes — `light_gathering_power`

How many times more light one telescope collects than another (ratio of aperture areas) and the corresponding gain in limiting magnitude.

$$\frac{P_1}{P_2} = \left(\frac{D_1}{D_2}\right)^2, \quad \Delta m = 5 \log_{10} \frac{D_1}{D_2}$$

input	meaning	condition
`D1`	aperture of the first telescope	
`D2`	aperture of the second telescope (or the eye's pupil)	

output	meaning
`area_ratio`	light-gathering ratio $(D_1/D_2)^2$
`mag_gain`	gain in limiting magnitude 5 log10(D1/D2)

- **Method:** ratio = (D1/D2)^2; gain = 2.5 log10(ratio)
- **Assumes:** D1, $D_2 > 0$; unobstructed circular apertures
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('light_gathering_power', D1=0.2, D2=0.002)
# -> area_ratio = 10000, mag_gain = 10   (SOLVED)
```

```
astronomy_ai999x.solve('light_gathering_power', D1=2.0, D2=1.0)
# -> area_ratio = 4, mag_gain = 1.50515   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Light-gathering power: D1 = 200 mm, D2 = 2 mm	SOLVED	area_ratio = 10000, mag_gain = 10
Light-gathering power: D1 = 200 mm, D2 = -2 mm	INVALID_INPUT	Light-gathering power of two telescopes: the two apertures must be a positive number.
Light-gathering power: D1 = 200 mm	NEEDS_INPUT	light_gathering_power needs D2 (aperture of the second telescope (or the eye's pupil)). Example: solve('light_gathering_

Pixel scale and field of view of a telescope camera — `camera_pixel_scale`

The angle on the sky covered by one detector pixel at the focal plane of a telescope and, for a row of N pixels, the field of view.

$$\theta = 2 \arctan \frac{w}{2f} \approx 206265'' \, \frac{w}{f}$$

input	meaning	condition
`f`	focal length of the telescope	
`p`	pixel size (um, mm)	
`N`	number of pixels across the detector (optional)	

output	meaning
`pixel_scale_arcsec`	pixel scale in arcseconds per pixel
`fov_deg`	field of view in degrees (with N)
`fov_arcmin`	field of view in arcminutes (with N)

- **Method:** angle = 2 atan(w / (2 f)) for a width w centred on the optical axis (about 206265 w/f arcseconds)
- **Assumes:** f, $p > 0$; $N \geq 1$; distortion-free optics; angles centred on the axis
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('camera_pixel_scale', f=0.5, p=1.0, N=1)
# -> pixel_scale_arcsec = 324000, fov_deg = 90, fov_arcmin = 5400   (SOLVED)
```

```
astronomy_ai999x.solve('camera_pixel_scale', f=2.0, p=3.76e-06)
# -> pixel_scale_arcsec = 0.387778   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pixel scale: f = 2000 mm, p = 3.76 um, N = 6248	SOLVED	pixel_scale_arcsec = 0.387778, fov_deg = 0.673002, fov_arcmin = 40.3801
Pixel scale: f = 2000 mm, p = 3.76 um, N = 0	INVALID_INPUT	N: must be at least 1, got 0.
Pixel scale: f = 2000 mm	NEEDS_INPUT	camera_pixel_scale needs p (pixel size (um, mm)). Example: solve('camera_pixel_scale', f='0.5 m', p='1 m', N=1).

Airmass and atmospheric extinction — `airmass_extinction`

The airmass X = sec z of a star at zenith angle z (plane-parallel atmosphere) and, with an extinction coefficient k, the extinction k X and the magnitude corrected to outside the atmosphere.

$$X = \sec z, \quad m_0 = m - kX$$

input	meaning	condition
`zen`	zenith angle (90 degrees minus the altitude)	
`k`	extinction coefficient in magnitudes per airmass (optional)	
`m`	observed magnitude (optional; needs k)	

output	meaning
`X`	airmass
`alt_deg`	altitude in degrees
`extinction`	extinction k X in magnitudes
`m0`	magnitude above the atmosphere m - k X

- **Method:** $X = 1/cos$ z; $m_0 = m - k$ X
- **Assumes:** $0 \le z \le 85$ degrees; plane-parallel atmosphere; $k \ge 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('airmass_extinction', zen=1.0471975511965976)
# -> X = 2, alt_deg = 30   (SOLVED)
```

```
astronomy_ai999x.solve('airmass_extinction', zen=1.0471975511965976, k=0.2, m=10.4)
# -> X = 2, alt_deg = 30, extinction = 0.4, m0 = 10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Airmass: zen = 60 deg	SOLVED	X = 2, alt_deg = 30
Airmass: zen = 60 deg, k = 0.2, m = 10.4	SOLVED	X = 2, alt_deg = 30, extinction = 0.4, m0 = 10
Airmass: zen = 89 deg	INVALID_INPUT	Airmass and atmospheric extinction: the zenith angle must lie between 0 and 85 degrees (the plane-parallel model fails n
Airmass:	NEEDS_INPUT	airmass_extinction needs zen (zenith angle (90 degrees minus the altitude)). Example: solve('airmass_extinction', zen='1

Altitude and azimuth from hour angle and declination — `altaz_from_hadec`

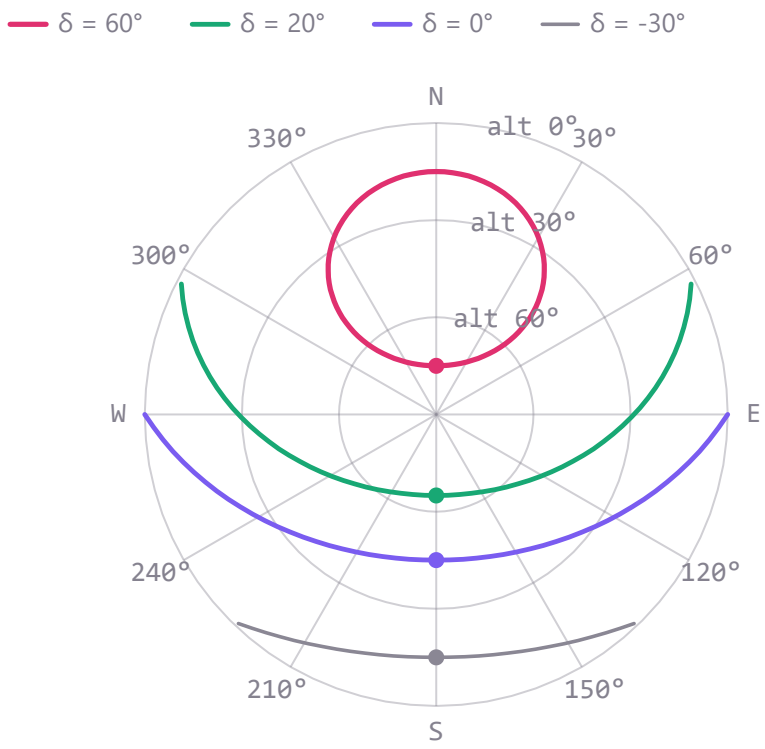
The altitude and azimuth of a star of declination dec at hour angle H seen from latitude lat (horizontal coordinates from equatorial ones).

$$\sin a = \sin\phi \sin\delta + \cos\phi \cos\delta \cos H, \quad \tan A = \frac{-\cos\delta \sin H}{\sin\delta \cos\phi - \cos\delta \sin\phi \cos H}$$

input	meaning	condition
`H`	hour angle (hours or degrees; positive west of the meridian)	
`dec`	declination	
`lat`	observer's latitude	

output	meaning
`alt_deg`	altitude in degrees (geometric, no refraction)
`az_deg`	azimuth in degrees from north through east

- **Method:** sin $a = sin$(lat) sin(dec) + cos(lat) cos(dec) cos H; tan $A = -cos$(dec) sin H / (sin(dec) cos(lat) - cos(dec) sin(lat) cos H)
- **Assumes:** |dec| <= 90 degrees; |lat| <= 90 degrees; not at the zenith or nadir; no refraction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sky chart: zenith at the centre, horizon at the rim

Daily paths across the sky of stars of four declinations, seen from latitude 45° north (azimuth around the rim, zenith distance outward; 97 hour angles each converted by the package to altitude and azimuth, only the part above the horizon drawn): δ = 60°: culminates at altitude 75°, azimuth 0°; δ = 20°: culminates at altitude 65°, azimuth 180°; δ = 0°: culminates at altitude 45°, azimuth 180°; δ = -30°: culminates at altitude 15°, azimuth 180°. The δ = 60° star circles the pole without setting.

```
astronomy_ai999x.solve('altaz_from_hadec', H=1.5707963267948966, dec=0.5235987755982988,
lat=0.7853981633974483)
# -> alt_deg = 20.7048, az_deg = 292.208   (SOLVED)
```

```
astronomy_ai999x.solve('altaz_from_hadec', H=0.0, dec=0.0, lat=0.7853981633974483)
# -> alt_deg = 45, az_deg = 180   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Altitude-azimuth from hour angle: H = 0 h, dec = 0 deg, lat = 45 deg	SOLVED	alt_deg = 45, az_deg = 180
Altitude-azimuth from hour angle: H = 6 h, dec = 0 deg, lat = 45 deg	SOLVED	alt_deg = 0, az_deg = 270
Altitude-azimuth from hour angle: H = 0 h, dec = 100 deg, lat = 45 deg	INVALID_INPUT	Altitude and azimuth from hour angle and declination: the declination must lie between -90 and 90 degrees.
Altitude-azimuth from hour angle: H = 0 h, dec = 10 deg	NEEDS_INPUT	altaz_from_hadec needs lat (observer's latitude). Example: solve('altaz_from_hadec', H='1.5708 rad', dec='0.523599 rad',

Altitude of a star at culmination — `culmination_altitude`

The altitudes of a star when it crosses the meridian above the pole (upper culmination) and below it (lower culmination), and whether it is circumpolar or never rises.

$$a_{\text{upper}} = 90^\circ - |\phi - \delta|, \quad a_{\text{lower}} = |\phi + \delta| - 90^\circ$$

input	meaning	condition
`dec`	declination	
`lat`	observer's latitude	

output	meaning
`alt_upper_deg`	altitude at upper culmination (degrees)
`alt_lower_deg`	altitude at lower culmination (degrees)
`circumpolar`	never sets
`never_rises`	never rises

- **Method:** upper: $a = 90$ - |lat - dec|; lower: a = |lat + dec| - 90
- **Assumes:** |dec|, |lat| <= 90 degrees; geometric horizon, no refraction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('culmination_altitude', dec=0.3490658503988659, lat=0.8726646259971648)
# -> alt_upper_deg = 60, alt_lower_deg = -20, circumpolar = False, never_rises = False   (SOLVED)
```

```
astronomy_ai999x.solve('culmination_altitude', dec=1.2217304763960306, lat=0.8726646259971648)
# -> alt_upper_deg = 70, alt_lower_deg = 30, circumpolar = True, never_rises = False   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Culmination altitude: dec = 20 deg, lat = 50 deg	SOLVED	alt_upper_deg = 60, alt_lower_deg = -20, circumpolar = False, never_rises = False
Culmination altitude: dec = -60 deg, lat = 50 deg	SOLVED	alt_upper_deg = -20, alt_lower_deg = -80, circumpolar = False, never_rises = True

question	outcome	result
Culmination altitude: dec = 20 deg, lat = 95 deg	INVALID_INPUT	Altitude of a star at culmination: the latitude must lie between -90 and 90 degrees.
Culmination altitude: dec = 20 deg	NEEDS_INPUT	culmination_altitude needs lat (observer's latitude). Example: solve('culmination_altitude', dec='0.349066 rad', lat='0.

Rising and setting of a star — `rise_set_hour_angle`

The hour angle at which a star sets, how long it stays above the horizon (sidereal hours) and the azimuths at which it rises and sets (geometric horizon, no refraction).

$$\cos H_0 = -\tan\phi\tan\delta, \quad \cos A_{\text{rise}} = \frac{\sin\delta}{\cos\phi}$$

input	meaning	condition
`dec`	declination	
`lat`	observer's latitude	

output	meaning
`H_set_h`	hour angle at setting (hours)
`time_up_sidereal_h`	time above the horizon (sidereal hours)
`az_rise_deg`	azimuth of rising (degrees from north through east)
`az_set_deg`	azimuth of setting

- **Method:** cos $H_0 = -tan$(lat) tan(dec); cos $A_{rise} = \frac{\sin(dec)}{\cos(lat)}$
- **Assumes:** the star rises and sets: |tan(lat) tan(dec)| < 1; geometric horizon (no refraction, no star size)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('rise_set_hour_angle', dec=0.0, lat=0.7853981633974483)
# -> H_set_h = 6, time_up_sidereal_h = 12, az_rise_deg = 90, az_set_deg = 270   (SOLVED)
```

```
astronomy_ai999x.solve('rise_set_hour_angle', dec=0.5235987755982988, lat=0.0)
# -> H_set_h = 6, time_up_sidereal_h = 12, az_rise_deg = 60, az_set_deg = 300   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rising and setting: dec = 0 deg, lat = 45 deg	SOLVED	H_set_h = 6, time_up_sidereal_h = 12, az_rise_deg = 90, az_set_deg = 270
Rising and setting: dec = 80 deg, lat = 50 deg	INVALID_INPUT	Rising and setting of a star: the star does not rise and set at this latitude (it is circumpolar or never rises).
Rising and setting: dec = 20 deg	NEEDS_INPUT	rise_set_hour_angle needs lat (observer's latitude). Example: solve('rise_set_hour_angle', dec='0 rad', lat='0.785398 ra

Angular separation of two sky positions — `angular_separation`

The great-circle angle between two objects given by their right ascensions and declinations.

$$\cos\theta = \sin\delta_1 \sin\delta_2 + \cos\delta_1 \cos\delta_2 \cos(\alpha_2 - \alpha_1)$$

input	meaning	condition
`ra1`	right ascension of object 1 (hours or degrees)	
`dec1`	declination of object 1	
`ra2`	right ascension of object 2	
`dec2`	declination of object 2	

output	meaning
`sep_deg`	separation in degrees
`sep_arcsec`	separation in arcseconds

- **Method:** Vincenty's formula (accurate at all separations)
- **Assumes:** |dec| <= 90 degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('angular_separation', ra1=0.0, dec1=0.0, ra2=1.5707963267948966, dec2=0.0)
# -> sep_deg = 90, sep_arcsec = 324000   (SOLVED)
```

```
astronomy_ai999x.solve('angular_separation', ra1=0.0, dec1=0.7853981633974483,
ra2=3.141592653589793, dec2=0.7853981633974483)
# -> sep_deg = 90, sep_arcsec = 324000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Angular separation: ra1 = 0 h, dec1 = 0 deg, ra2 = 6 h, dec2 = 0 deg	SOLVED	sep_deg = 90, sep_arcsec = 324000
Angular separation: ra1 = 0 h, dec1 = 95 deg, ra2 = 6 h, dec2 = 0 deg	INVALID_INPUT	Angular separation of two sky positions: declinations must lie between -90 and 90 degrees.
Angular separation: ra1 = 0 h, dec1 = 0 deg	NEEDS_INPUT	angular_separation needs ra2, dec2 (right ascension of object 2; declination of object 2). Example: solve('angular_separ

Position angle between two sky positions — `position_angle`

The position angle of object 2 as seen from object 1, measured from north through east.

$$\tan \mathrm{PA} = \frac{\sin\Delta\alpha \cos\delta_2}{\cos\delta_1 \sin\delta_2 - \sin\delta_1 \cos\delta_2 \cos\Delta\alpha}$$

input	meaning	condition
`ra1`	right ascension of object 1 (hours or degrees)	

input	meaning	condition
`dec1`	declination of object 1	
`ra2`	right ascension of object 2	
`dec2`	declination of object 2	

output	meaning
`pa_deg`	position angle in degrees (0 = north, 90 = east)

- **Method:** tan $PA = sin$(dra) cos(dec2) / (cos(dec1) sin(dec2) - sin(dec1) cos(dec2) cos(dra))
- **Assumes:** |dec| < 90 degrees; distinct, non-antipodal positions
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('position_angle', ra1=0.0, dec1=0.0, ra2=0.0, dec2=0.1)
# -> pa_deg = 0   (SOLVED)
```

```
astronomy_ai999x.solve('position_angle', ra1=0.0, dec1=0.0, ra2=0.1, dec2=0.0)
# -> pa_deg = 90   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Position angle: ra1 = 0 deg, dec1 = 0 deg, ra2 = 1 deg, dec2 = 0 deg	SOLVED	pa_deg = 90
Position angle: ra1 = 0 deg, dec1 = 0 deg, ra2 = 0 deg, dec2 = 0 deg	INVALID_INPUT	Position angle between two sky positions: the two positions coincide (or are antipodal): no position angle.
Position angle: ra1 = 0 deg, dec1 = 0 deg	NEEDS_INPUT	position_angle needs ra2, dec2 (right ascension of object 2; declination of object 2). Example: solve('position_angle',

Julian date of a calendar date — `julian_date`

The Julian date and the modified Julian date of a Gregorian calendar date and time of day (in the same time scale as the date, e.g. TT or UT).

$$\mathrm{JD} = \lfloor 365.25(y+4716) \rfloor + \lfloor 30.6001(m+1) \rfloor + D + B - 1524.5$$

input	meaning	condition
`year`	year (Gregorian, 1583 or later)	
`month`	month $(1-12)$	
`day`	day of the month	
`hour`	hour (0-23, default 0)	
`minute`	minute (0-59, default 0)	
`second`	second (0 <= s < 60, default 0)	

output	meaning
`JD`	Julian date (days)

output	meaning
`MJD`	modified Julian date $JD - 2400000.5$

- **Method:** Meeus' algorithm in exact integer arithmetic: JD = floor(365.25(y+4716)) + floor(30.6001(m+1)) + $D + B - 1524.5$
- **Assumes:** Gregorian calendar (on or after 1582-10-15); a valid date and time of day (no leap second)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('julian_date', year=2000, month=1, day=1, hour=12)
# -> JD = 2.45154e+06, MJD = 51544.5   (SOLVED)
```

```
astronomy_ai999x.solve('julian_date', year=1957, month=10, day=4, hour=19, minute=26, second=24)
# -> JD = 2.43612e+06, MJD = 36115.8   (SOLVED)
```

```
astronomy_ai999x.solve('julian_date', year=1858, month=11, day=18)
# -> JD = 2.4e+06, MJD = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Julian date: year = 2000, month = 1, day = 1, hour = 12	SOLVED	JD = 2.45154e+06, MJD = 51544.5
Julian date: year = 2023, month = 2, day = 29	INVALID_INPUT	Julian date of a calendar date: day must be 1 to 28 in that month.
Julian date: year = 2000, month = 1	NEEDS_INPUT	julian_date needs day (day of the month). Example: solve('julian_date', year=2000, month=1, day=1, hour=12).

Calendar date from a Julian date — `calendar_date_from_julian_date`

The Gregorian calendar date and the time of day (decimal hours) of a Julian date.

$$Z = \lfloor \mathrm{JD} + 0.5 \rfloor,\ F = \mathrm{JD} + 0.5 - Z$$

input	meaning	condition
`jd`	Julian date (days)	

output	meaning
`year`	year
`month`	month
`day`	day of the month
`hours`	time of day in decimal hours

- **Method:** Meeus' inverse algorithm in exact integer and rational arithmetic
- **Assumes:** $2299160.5 \le JD \le 5373484.5$ (Gregorian era up to the year 9999)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('calendar_date_from_julian_date', jd=2451545.0)
# -> year = 2000, month = 1, day = 1, hours = 12   (SOLVED)
```

```
astronomy_ai999x.solve('calendar_date_from_julian_date', jd=2436116.31)
# -> year = 1957, month = 10, day = 4, hours = 19.44   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Calendar date from Julian date: jd = 2451545.0	SOLVED	year = 2000, month = 1, day = 1, hours = 12
Calendar date from Julian date: jd = 1000000	INVALID_INPUT	Calendar date from a Julian date: the Julian date must lie in the Gregorian era (JD 2299160.5 to 5373484.5).
Calendar date from Julian date:	NEEDS_INPUT	calendar_date_from_julian_date needs jd (Julian date (days)). Example: solve('calendar_date_from_julian_date', jd=2.4515

Greenwich and local mean sidereal time — `sidereal_time`

The Greenwich mean sidereal time (IAU 1982 expression) at a Julian date in UT1, and the local mean sidereal time at an east longitude.

$$\mathrm{GMST} = 24110.54841^{\mathrm{s}} + 8640184.812866^{\mathrm{s}}\,T + 0.093104^{\mathrm{s}}\,T^2 - 6.2\times10^{-6\,\mathrm{s}}\,T^3 + \mathrm{UT1}$$

input	meaning	condition
`jd`	Julian date (UT1)	
`lon`	east longitude (default 0)	

output	meaning
`gmst_h`	Greenwich mean sidereal time (hours)
`lmst_h`	local mean sidereal time (hours)

- **Method:** GMST = 24110.54841 $s + 8640184.812866\,T + 0.093104\,T^2 - 6.2e-6\,T^3 + UT1$ seconds of the day (T from J2000, Julian centuries); LMST = GMST + longitude/15
- **Assumes:** $2299160.5 \le JD \le 5373484.5$; the date in UT1; east longitudes positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('sidereal_time', jd=2451545.0)
# -> gmst_h = 18.6974, lmst_h = 18.6974   (SOLVED)
```

```
astronomy_ai999x.solve('sidereal_time', jd=2451545.0, lon=0.2617993877991494)
# -> gmst_h = 18.6974, lmst_h = 19.6974   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sidereal time: jd = 2451545.0	SOLVED	gmst_h = 18.6974, lmst_h = 18.6974
Sidereal time: jd = 100	INVALID_INPUT	Greenwich and local mean sidereal time: the Julian date must lie between 2299160.5 and 5373484.5.
Sidereal time:	NEEDS_INPUT	sidereal_time needs jd (Julian date (UT1)). Example: solve('sidereal_time', jd=2.45154e+06).

Plasma frequency — `plasma_frequency`

The plasma (Langmuir) oscillation frequency of electrons (or protons) of number density n.

$$\omega_p = \sqrt{\frac{ne^2}{\varepsilon_0 m}}$$

input	meaning	condition
`n`	number density (m^-3, cm^-3)	
`particle`	particle species: electron or proton (default electron)	

output	meaning
`omega_p`	plasma angular frequency
`f_p`	plasma frequency

- **Method:** $omega_p = sqrt$(n e^2 / (eps0 m))
- **Assumes:** $n > 0$; cold, unmagnetized plasma; singly charged particles
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('plasma_frequency', n=0.0003142077833705378)
# -> omega_p = 1 rad/s, f_p = 0.159155 Hz   (SOLVED)
```

```
astronomy_ai999x.solve('plasma_frequency', n=2.3077338457826597, particle='proton')
# -> omega_p = 2 rad/s, f_p = 0.31831 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plasma frequency: n = 1e12 m^-3	SOLVED	omega_p = 5.64146e+07 rad/s, f_p = 8.97866e+06 Hz
Plasma frequency: n = 1e12 m^-3, particle = neutron	INVALID_INPUT	particle: one of electron, proton; got 'neutron'.
Plasma frequency:	NEEDS_INPUT	plasma_frequency needs n (number density (m^-3, cm^-3)). Example: solve('plasma_frequency', n='0.000314208 1/m^3').

Debye length — `debye_length`

The electron Debye screening length of a plasma of electron temperature T and electron density n.

$$\lambda_D = \sqrt{\frac{\varepsilon_0 k_B T_e}{n_e e^2}}$$

input	meaning	condition
`T`	electron temperature	
`n`	electron density (m^-3, cm^-3)	

output	meaning
`lambda_D`	Debye length
`lambda_D_nm`	Debye length in nanometres

- **Method:** $lambda_D = sqrt$(eps0 k T / (n e^2))
- **Assumes:** T, $n > 0$; electron screening only
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('debye_length', T=0.00020998524273112756, n=1.0)
# -> lambda_D = 1 m, lambda_D_nm = 1e+09   (SOLVED)
```

```
astronomy_ai999x.solve('debye_length', T=0.0008399409709245102, n=1.0)
# -> lambda_D = 2 m, lambda_D_nm = 2e+09   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Debye length: T = 1e5 K, n = 1e18 m^-3	SOLVED	lambda_D = 2.18226e-05 m, lambda_D_nm = 21822.6
Debye length: T = 1e5 K, n = 0 m^-3	INVALID_INPUT	Debye length: the temperature and the density must be a positive number.
Debye length: T = 1e5 K	NEEDS_INPUT	debye_length needs n (electron density (m^-3, cm^-3)). Example: solve('debye_length', T='0.000209985 K', n='1 1/m^3').

Number of particles in a Debye sphere — `debye_number`

The number of electrons in a sphere of one Debye length radius (N_D >> 1 means the gas behaves as a plasma).

$$N_D = \frac{4\pi}{3} n_e \lambda_D^3$$

input	meaning	condition
`T`	electron temperature	
`n`	electron density	

output	meaning
`N_D`	particles per Debye sphere

output	meaning
`lambda_D`	Debye length

- **Method:** N_D = (4 pi / 3) n $lambda_D^3$
- **Assumes:** T, $n > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('debye_number', T=0.00020998524273112756, n=1.0)
# -> N_D = 4.18879, lambda_D = 1 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Debye number: T = 1e6 K, n = 1e15 m^-3	SOLVED	N_D = 4.35317e+07, lambda_D = 0.00218226 m
Debye number: T = -1 K, n = 1e15 m^-3	INVALID_INPUT	Number of particles in a Debye sphere: the temperature and the density must be a positive number.
Debye number: n = 1e15 m^-3	NEEDS_INPUT	debye_number needs T (electron temperature). Example: solve('debye_number', T='0.000209985 K', n='1 1/m^3').

Alfven speed — `alfven_speed`

The Alfven speed of a fully ionized hydrogen plasma (n protons and n electrons per unit volume) in a magnetic field B.

$$v_A = \frac{B}{\sqrt{\mu_0\, n\, (m_p + m_e)}}$$

input	meaning	condition
`B`	magnetic field strength (T, nT, gauss)	
`n`	proton number density	

output	meaning
`v_A`	Alfven speed
`v_A_km_s`	Alfven speed in km/s

- **Method:** $v_A = B/sqrt$(mu0 n (m_p + m_e))
- **Assumes:** B, $n > 0$; mass density n (m_p + m_e); non-relativistic (v_A << c)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('alfven_speed', B=4.5858733307403914e-17, n=1.0)
# -> v_A = 1 m/s, v_A_km_s = 0.001   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Alfven speed: B = 5 nT, n = 5 cm^-3	SOLVED	v_A = 48759.9 m/s, v_A_km_s = 48.7599
Alfven speed: B = 0 nT, n = 5 cm^-3	INVALID_INPUT	Alfven speed: the field and the density must be a positive number.
Alfven speed: B = 5 nT	NEEDS_INPUT	alfven_speed needs n (proton number density). Example: solve('alfven_speed', B='4.58587e-17 T', n='1 1/m^3').

Gyrofrequency (cyclotron frequency) — `gyrofrequency`

The angular and ordinary gyrofrequency and the gyration period of an electron or proton in a magnetic field B.

$$\omega_c = \frac{|q|B}{m}$$

input	meaning	condition
`B`	magnetic field strength (T, nT, gauss)	
`particle`	particle species: electron or proton (default electron)	

output	meaning
`omega_c`	angular gyrofrequency
`f_c`	gyrofrequency
`T_c`	gyration period

- **Method:** omega_c = |q| B / m
- **Assumes:** $B > 0$; non-relativistic particle
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('gyrofrequency', B=5.685630111305193e-12)
# -> omega_c = 1 rad/s, f_c = 0.159155 Hz, T_c = 6.28319 s   (SOLVED)
```

```
astronomy_ai999x.solve('gyrofrequency', B=1.0439684928958963e-08, particle='proton')
# -> omega_c = 1 rad/s, f_c = 0.159155 Hz, T_c = 6.28319 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gyrofrequency: B = 1 T, particle = proton	SOLVED	omega_c = 9.57883e+07 rad/s, f_c = 1.52452e+07 Hz, T_c = 6.55945e-08 s
Gyrofrequency: B = -1 T	INVALID_INPUT	Gyrofrequency (cyclotron frequency): the field must be a positive number.
Gyrofrequency: particle = proton	NEEDS_INPUT	gyrofrequency needs B (magnetic field strength (T, nT, gauss)). Example: solve('gyrofrequency', B='5.68563e-12 T').

Gyroradius (Larmor radius) — `gyroradius`

The radius of the circular motion of an electron or proton moving with perpendicular speed v_perp in a magnetic field B.

$$r_L = \frac{m v_\perp}{|q| B}$$

input	meaning	condition
`B`	magnetic field strength	
`v_perp`	speed perpendicular to the field	
`particle`	particle species: electron or proton (default electron)	

output	meaning
`r_L`	gyroradius
`r_L_km`	gyroradius in kilometres
`r_L_nm`	gyroradius in nanometres

- **Method:** $r_L = m$ v_perp / (|q| B)
- **Assumes:** B, v_perp > 0; v_perp < 0.1 c (non-relativistic)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('gyroradius', B=1.0439684928958963e-08, v_perp=3.0, particle='proton')
# -> r_L = 3 m, r_L_km = 0.003, r_L_nm = 3e+09   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gyroradius: B = 5 nT, v_perp = 400 km/s, particle = proton	SOLVED	r_L = 835175 m, r_L_km = 835.175, r_L_nm = 8.35175e+14
Gyroradius: B = 5 nT, v_perp = 2e8 m/s	INVALID_INPUT	Gyroradius (Larmor radius): the speed must be well below c (v < 0.1 c) for the non-relativistic gyroradius.
Gyroradius: B = 5 nT	NEEDS_INPUT	gyroradius needs v_perp (speed perpendicular to the field). Example: solve('gyroradius', B='1.04397e-08 T', v_perp='3 m/

Thermal speed of plasma particles — `thermal_speed`

The thermal speed of electrons or protons of temperature T under a stated convention: most probable sqrt(2kT/m), root-mean-square sqrt(3kT/m) or mean sqrt(8kT/(pi m)).

$$v_{\mathrm{mp}} = \sqrt{\frac{2kT}{m}},\ v_{\mathrm{rms}} = \sqrt{\frac{3kT}{m}},\ \bar{v} = \sqrt{\frac{8kT}{\pi m}}$$

input	meaning	condition
`T`	temperature	
`convention`	which thermal speed: most_probable, rms or mean	

input	meaning	condition
`particle`	particle species: electron or proton (default electron)	

output	meaning
`v_th`	thermal speed

- **Method:** $v = sqrt$(c k T / m) with $c = 2$ (most probable), 3 (rms), $8/pi$ (mean)
- **Assumes:** $T > 0$; Maxwellian distribution in three dimensions; the convention must be stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('thermal_speed', T=6.597899765907193e-08, convention='rms')
# -> v_th = 1.73205 m/s   (SOLVED)
```

```
astronomy_ai999x.solve('thermal_speed', T=0.00012114751294137755, convention='most_probable',
particle='proton')
# -> v_th = 1.41421 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thermal speed: T = 1e6 K, convention = rms, particle = proton	SOLVED	v_th = 157363 m/s
Thermal speed: T = 1e6 K, convention = fastest	INVALID_INPUT	convention: one of most_probable, rms, mean; got 'fastest'.
Thermal speed: T = 1e6 K	NEEDS_INPUT	thermal_speed needs convention (which thermal speed: most_probable, rms or mean). Example: solve('thermal_speed', T='6.5

Plasma beta — `plasma_beta`

The ratio of the thermal pressure n k T of one particle species to the magnetic pressure B^2/(2 mu0).

$$\beta = \frac{n k_B T}{B^2/(2\mu_0)}$$

input	meaning	condition
`T`	temperature of the species	
`n`	number density of the species	
`B`	magnetic field strength	

output	meaning
`beta`	plasma beta
`p_thermal`	thermal pressure n k T
`p_magnetic`	magnetic pressure

- **Method:** $\beta = n$ k T / (B^2 / (2 mu0))
- **Assumes:** T, n, $B > 0$; one species (add species' pressures for a total beta)

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('plasma_beta', T=7.24297051603992e+22, n=1.0, B=0.0015853309189377467)
# -> beta = 1, p_thermal = 1 Pa, p_magnetic = 1 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plasma beta: T = 1e5 K, n = 5 cm^-3, B = 5 nT	SOLVED	beta = 0.69399, p_thermal = 6.90325e-12 Pa, p_magnetic = 9.94718e-12 Pa
Plasma beta: T = 1e5 K, n = 5 cm^-3, B = 0 nT	INVALID_INPUT	Plasma beta: the temperature, the density and the field must be a positive number.
Plasma beta: T = 1e5 K, n = 5 cm^-3	NEEDS_INPUT	plasma_beta needs B (magnetic field strength). Example: solve('plasma_beta', T='7.24297e+22 K', n='1 1/m^3', B='0.001585

Magnetic pressure — `magnetic_pressure`

The magnetic pressure (equal to the magnetic energy density) of a field B in vacuum or a plasma.

$$p_B = \frac{B^2}{2\mu_0}$$

input	meaning	condition
`B`	magnetic field strength (T, nT, gauss)	

output	meaning
`p_B`	magnetic pressure
`energy_density`	magnetic energy density (J/m^3)
`p_B_nPa`	magnetic pressure in nanopascals

- **Method:** p_B = B^2 / (2 mu0)
- **Assumes:** $B > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('magnetic_pressure', B=0.0015853309189377467)
# -> p_B = 1 Pa, energy_density = 1 Pa, p_B_nPa = 1e+09   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Magnetic pressure: B = 1 T	SOLVED	p_B = 397887 Pa, energy_density = 397887 Pa, p_B_nPa = 3.97887e+14
Magnetic pressure: B = 0 T	INVALID_INPUT	Magnetic pressure: the field must be a positive number.
Magnetic pressure:	NEEDS_INPUT	magnetic_pressure needs B (magnetic field strength (T, nT, gauss)). Example: solve('magnetic_pressure', B='0.00158533 T'

Upper hybrid frequency — `upper_hybrid_frequency`

The upper hybrid frequency of a magnetized electron plasma: sqrt(omega_pe^2 + omega_ce^2).

$$\omega_{uh} = \sqrt{\omega_{pe}^2 + \omega_{ce}^2}$$

input	meaning	condition
`B`	magnetic field strength	
`n`	electron density	

output	meaning
`omega_uh`	upper hybrid angular frequency
`f_uh`	upper hybrid frequency

- **Method:** $\omega_{uh} = \sqrt{\omega_{pe}^2 + \omega_{ce}^2}$
- **Assumes:** B, $n > 0$; cold plasma, waves perpendicular to B
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('upper_hybrid_frequency', B=1.7056890333915583e-11, n=0.005027324533928605)
# -> omega_uh = 5 rad/s, f_uh = 0.795775 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Upper hybrid frequency: B = 1 T, n = 1e18 m^-3	SOLVED	omega_uh = 1.84708e+11 rad/s, f_uh = 2.93972e+10 Hz
Upper hybrid frequency: B = 1 T, n = -1e18 m^-3	INVALID_INPUT	Upper hybrid frequency: the field and the density must be a positive number.
Upper hybrid frequency: B = 1 T	NEEDS_INPUT	upper_hybrid_frequency needs n (electron density). Example: solve('upper_hybrid_frequency', B='1.70569e-11 T', n='0.0050

Lower hybrid frequency — `lower_hybrid_frequency`

The lower hybrid frequency of a magnetized electron-proton plasma.

$$\frac{1}{\omega_{lh}^2} = \frac{1}{\omega_{ci}\,\omega_{ce}} + \frac{1}{\omega_{pi}^2}$$

input	meaning	condition
`B`	magnetic field strength	
`n`	density (electrons = protons)	

output	meaning
`omega_lh`	lower hybrid angular frequency
`f_lh`	lower hybrid frequency

- **Method:** 1/omega_lh^2 = 1/(omega_ci omega_ce) + 1/omega_pi^2
- **Assumes:** B, $n > 0$; cold quasi-neutral electron-proton plasma
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('lower_hybrid_frequency', B=1.0, n=9.719853830824368e+18)
# -> omega_lh = 2.90236e+09 rad/s, f_lh = 4.61926e+08 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lower hybrid frequency: B = 1 T, n = 1e18 m^-3	SOLVED	omega_lh = 1.25364e+09 rad/s, f_lh = 1.99523e+08 Hz
Lower hybrid frequency: B = 1 T, n = 0 m^-3	INVALID_INPUT	Lower hybrid frequency: the field and the density must be a positive number.
Lower hybrid frequency: B = 1 T	NEEDS_INPUT	lower_hybrid_frequency needs n (density (electrons = protons)). Example: solve('lower_hybrid_frequency', B='1 T', n='9.7

Inertial length (skin depth) — `inertial_length`

The electron (or proton) inertial length c / omega_p, the collisionless skin depth of a plasma.

$$d = \frac{c}{\omega_p}$$

input	meaning	condition
`n`	number density	
`particle`	particle species: electron or proton (default electron)	

output	meaning
`d`	inertial length
`d_km`	inertial length in kilometres

- **Method:** $d = c/omega_p = c$ sqrt(eps0 m / (n e^2))
- **Assumes:** $n > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('inertial_length', n=28239587250368.695)
# -> d = 1 m, d_km = 0.001   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Inertial length: n = 5 cm^-3, particle = proton	SOLVED	d = 101835 m, d_km = 101.835
Inertial length: n = 0 cm^-3	INVALID_INPUT	Inertial length (skin depth): the density must be a positive number.

question	outcome	result
Inertial length:	NEEDS_INPUT	inertial_length needs n (number density). Example: solve('inertial_length', n='2.82396e+13 1/m^3').

Einstein radius of a point-mass lens — `einstein_radius`

The angular Einstein radius of a point mass M at distance D_l lensing a source at distance D_s (static space: D_ls = D_s - D_l), and the corresponding radius in the lens plane.

$$\theta_E = \sqrt{\frac{4GM}{c^2}\frac{D_{ls}}{D_l D_s}}$$

input	meaning	condition
`M`	lens mass (Msun, kg)	
`D_l`	distance to the lens (pc, kpc)	
`D_s`	distance to the source	

output	meaning
`theta_E`	Einstein angle
`theta_E_arcsec`	Einstein angle in arcseconds
`R_E`	Einstein radius in the lens plane
`R_E_au`	Einstein radius in au

- **Method:** $theta_E = sqrt$((4 G M / c^2) (D_s - D_l) / (D_l D_s))
- **Assumes:** $D_s > D_l > 0$; point mass, weak field; non-cosmological distances $(D_{ls} = D_s - D_l)$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('einstein_radius', M=3.3664773037502725e+26, D_l=1.0, D_s=2.0)
# -> theta_E = 0.707107 rad, theta_E_arcsec = 145851, R_E = 0.707107 m, R_E_au = 4.72672e-12
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Einstein radius: M = 0.5 Msun, D_l = 4 kpc, D_s = 8 kpc	SOLVED	theta_E = 3.45884e-09 rad, theta_E_arcsec = 0.000713436, R_E = 4.26914e+11 m, R_E_au = 2.85374
Einstein radius: M = 0.5 Msun, D_l = 8 kpc, D_s = 4 kpc	INVALID_INPUT	Einstein radius of a point-mass lens: the source must lie behind the lens (D_s > D_l).
Einstein radius: M = 0.5 Msun, D_l = 4 kpc	NEEDS_INPUT	einstein_radius needs D_s (distance to the source). Example: solve('einstein_radius', M='3.36648e+26 kg', D_l='1 m', D_s

Distance from luminosity and flux (inverse-square law) — `distance_from_flux`

The distance of a source of known luminosity L (a standard candle) from the bolometric flux F measured at the observer.

$$d = \sqrt{\frac{L}{4\pi F}}$$

input	meaning	condition
`L`	luminosity (W or Lsun)	
`F`	measured flux (W/m^2)	

output	meaning
`d`	distance
`d_pc`	distance in parsecs
`d_ly`	distance in light-years

- **Method:** F = L / (4 pi d^2): $d = sqrt$(L / (4 pi F))
- **Assumes:** L, $F > 0$; isotropic emission, no extinction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('distance_from_flux', L=12.566370614359172, F=1.0)
# -> d = 1 m, d_pc = 3.24078e-17, d_ly = 1.057e-16   (SOLVED)
```

```
astronomy_ai999x.solve('distance_from_flux', L=3.0128e+28, F=2.518021002633401e-10)
# -> d = 3.08568e+18 m, d_pc = 100, d_ly = 326.156   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Distance from flux: L = 1 Lsun, F = 1361 W/m^2	SOLVED	d = 1.49607e+11 m, d_pc = 4.84843e-06, d_ly = 1.58135e-05
Distance from flux: L = 1 Lsun, F = 0 W/m^2	INVALID_INPUT	Distance from luminosity and flux (inverse-square law): the luminosity and the flux must be a positive number.
Distance from flux: L = 1 Lsun	NEEDS_INPUT	distance_from_flux needs F (measured flux (W/m^2)). Example: solve('distance_from_flux', L='12.5664 W', F='1 W/m^2').

Hubble parameter and critical density — `critical_density`

The Hubble parameter H(z) and the critical density 3H^2/(8 pi G) of a flat Lambda-CDM universe at redshift z (z = 0 gives today's values), with the equivalent number of hydrogen atoms per cubic metre.

$$H(z) = H_0\sqrt{\Omega_m(1+z)^3 + \Omega_\Lambda}, \quad \rho_c = \frac{3H^2}{8\pi G}$$

input	meaning	condition
`H0`	Hubble constant in km/s/Mpc	
`Om`	matter density parameter Omega_m	
`z`	redshift (default 0)	

output	meaning
`H_z`	Hubble parameter H(z) in km/s/Mpc
`rho_c`	critical density
`n_H`	critical density in hydrogen-atom (proton) masses per cubic metre

- **Method:** H(z) = H0 sqrt(Om (1+z)^3 + 1 - Om); $rho_c = 3$ H^2 / (8 pi G)
- **Assumes:** $1 \leq H_0 \leq 1000$; $0.01 \leq Omega_m \leq 1$; $0 \leq z \leq 1100$; flat, no radiation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
astronomy_ai999x.solve('critical_density', H0=70.0, Om=0.3)
# -> H_z = 70, rho_c = 9.20387e-27 kg/m^3, n_H = 5.50266 1/m^3   (SOLVED)
```

```
astronomy_ai999x.solve('critical_density', H0=70.0, Om=1.0, z=3.0)
# -> H_z = 560, rho_c = 5.89048e-25 kg/m^3, n_H = 352.17 1/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical density: H0 = 70 km/s/Mpc, Om = 0.3	SOLVED	H_z = 70, rho_c = 9.20387e-27 kg/m^3, n_H = 5.50266 1/m^3
Critical density: H0 = 70, Om = 0.3, z = -2	INVALID_INPUT	Hubble parameter and critical density: the redshift must lie between 0 and 1100.
Critical density: H0 = 70	NEEDS_INPUT	critical_density needs Om (matter density parameter Omega_m). Example: solve('critical_density', H0=70, Om=0.3).